Autonomous and Mobile Robotics

Lecture Notes

Flavio Massaroni

Year 2026

Sapienza Università di Roma
Dipartimento di Ingegneria Informatica, Automatica e Gestionale

Preface

These notes were written during the *Autonomous and Mobile Robotics* course held by Prof. Giuseppe Oriolo at Sapienza University of Rome. The main purpose of this document is to provide a coherent and compact reference for the topics covered: kinematics of mobile robots, motion planning, motion control, localization, mapping, SLAM, and decision making under uncertainty. Theoretical derivations are interspersed with practical remarks, formulas, and, where useful, pseudocode or short implementations.

October 29, 2026

Contents

Preface ii

1 Introduction: Applications, Problems, Architectures 1
- 1.1 Why Mobile Robots . . . 1
- 1.2 Applications . . . 1
- 1.3 The Key Problems of Mobile Robotics . . . 1
- 1.4 Autonomy . . . 2
- 1.5 Perception . . . 2
- 1.6 Robot Architectures . . . 2
  - 1.6.1 Deliberative Architecture . . . 2
  - 1.6.2 Reactive Architecture . . . 3
  - 1.6.3 Hybrid Architectures . . . 3
  - 1.6.4 Behavior-Based Architecture . . . 3
- 1.7 Course Approach . . . 4

2 Configuration Space 5
- 2.1 Motivation: Why Configuration Space? . . . 5
- 2.2 Modeling a Robot . . . 5
  - 2.2.1 Configuration and Degrees of Freedom . . . 5
- 2.3 The Configuration Space and the Workspace . . . 6
  - 2.3.1 Workspace vs Configuration Space . . . 7
- 2.4 Examples . . . 7
  - 2.4.1 Point Robot . . . 7
  - 2.4.2 Disk Robot . . . 7
  - 2.4.3 Robot with Orientation: $\mathbb{R}^2 \times \mathcal{S}^1$ . . . 8
  - 2.4.4 Planar 2-Link Manipulator: $\mathcal{T}^2$ . . . 8
  - 2.4.5 Rigid Body in 3D: $SE(3)$ . . . 8
- 2.5 Topology of Configuration Space . . . 8
  - 2.5.1 The 2R Planar Manipulator . . . 8
  - 2.5.2 Non-Injectivity of the Parameterization . . . 8
  - 2.5.3 Restoring Injectivity: the Fundamental Domain . . . 9
  - 2.5.4 The Remaining Problem: Closeness Is Not Preserved . . . 9
  - 2.5.5 Gluing the Square: Why C Is a Torus . . . 10
  - 2.5.6 Why Topology Matters . . . 10
- 2.6 Distance in the Configuration Space . . . 11
  - 2.6.1 Geodesic Distance . . . 11
  - 2.6.2 The Displacement Metric . . . 12
  - 2.6.3 Control Points . . . 12
- 2.7 C-Obstacles and Free Space . . . 13
- 2.8 Relevance to Motion Planning . . . 13

3 Wheeled Mobile Robots: Mechanics **14**
3.1 Sources of Locomotion . . . 14
3.2 Static Balance . . . 14
3.3 Types of Wheels . . . 15
3.4 Kinematic Structures . . . 15
3.4.1 Differential Drive . . . 15
3.4.2 Synchro-Drive . . . 16
3.4.3 Tricycle and Car-Like . . . 16
3.5 The Differential . . . 16
3.6 Constraints . . . 17
3.6.1 Geometric Constraints . . . 17
3.6.2 Kinematic Constraints . . . 17
3.6.3 Holonomic vs Nonholonomic Constraints . . . 18
3.7 Kinematic Models . . . 19
3.7.1 Controllability and the Accessibility Rank Condition . . . 20
3.7.2 The Unicycle Model . . . 21
3.7.3 The Bicycle Model . . . 22
3.8 Chained Form . . . 24
3.8.1 The Unicycle in Chained Form . . . 25
3.8.2 The Bicycle in Chained Form . . . 25
3.9 Takeaways . . . 27

4 Nonlinear Controllability in Depth **28**
4.1 Scope of This Chapter . . . 28
4.2 Controllability of Driftless Systems . . . 28
4.3 Lie Brackets in Depth . . . 29
4.4 Interpretation of the Lie Bracket . . . 30
4.5 Distributions and Integrability . . . 31
4.6 Exercises . . . 32
4.7 Takeaways . . . 33

5 Path and Trajectory Planning **34**
5.1 Path vs Trajectory . . . 34
5.2 The Geometric Model . . . 35
5.3 Differential Flatness . . . 36
5.4 Path Planning for the Unicycle . . . 37
5.5 Path Planning for a Chained-Form System . . . 39
5.6 Path Planning via Parametrized Inputs . . . 39
5.7 Choice of the Timing Law . . . 40
5.7.1 Linear Timing Law . . . 40
5.7.2 Trapezoidal Timing Law . . . 40
5.7.3 Velocity and Acceleration Bounds . . . 40
5.8 Takeaways . . . 41

6 Motion Control **42**
6.1 Trajectory Tracking for the Unicycle . . . 43
6.1.1 Error Dynamics . . . 43
6.1.2 Approximate Linearization . . . 44
6.1.3 Output Error Feedback: Static . . . 46
6.1.4 Output Error Feedback: Dynamic . . . 47
6.1.5 Comparison of the Controllers . . . 48
6.2 Takeaways . . . 48

7 Localizzazione **49**

8 Mapping **50**

9 SLAM **51**

10 Decisioni incertezza **52**

11 Laboratorio **53**

A Formulario **54**

B Esercizi **55**

C Codice **56**
C.1 Esempio: EKF SLAM 56
C.2 Esempio: RRT 56

List of Tables

1.1 Comparison between fixed-base manipulators and wheeled mobile robots. The apparent simplicity of WMRs hides fundamental difficulties. 2

Chapter 1

Introduction: Applications, Problems, Architectures

This introductory chapter frames the course. It does not introduce technical tools yet; its purpose is to define *what* mobile robotics is about, *why* it is hard, and *how* we will approach it.

1.1 Why Mobile Robots

Industrial fixed-base manipulators are fast and accurate, but they operate in a *limited, structured, known, static* workspace. To be useful outside the factory, robots must move in *large, unstructured, uncertain, dynamic* environments. This shift is what motivates the field of autonomous and mobile robotics.

The course focuses on *methods*, not on specific hardware or software: the goal is to master the basic planning and control techniques that enable mobility and autonomy on *any* mobile platform — wheeled, legged, flying, or underwater.

1.2 Applications

Mobile robots are already deployed in a wide spectrum of domains, which can be roughly divided into:

- **Structured environments (service robots)**: cleaning, catering, customer assistance, surveillance, entertainment.
- **Unstructured environments (field robots)**: planetary exploration, underwater monitoring, rescue, demining, agriculture, construction, military transportation.

The market for autonomous mobile robots is growing rapidly, with goods-to-person picking robots, self-driving forklifts, autonomous inventory robots, and unmanned aerial vehicles among the main segments.

1.3 The Key Problems of Mobile Robotics

Following Durrant-Whyte (1991), the field can be organized around three fundamental questions:

1. **Where am I?** — *localization*
2. **How am I supposed to get to the goal?** — *path / trajectory / motion planning*
3. **How do I actually move?** — *motion control*

The entire course is a declination of these three problems.

Why wheeled mobile robots are harder than they look. A useful comparison is between fixed-base manipulators and single-body wheeled mobile robots (WMRs):

	Fixed-base manipulator	**WMR**
1. Localization	easy (joint encoders)	difficult
2a. Path/trajectory planning	easy (all paths feasible)	difficult (not all paths feasible)
2b. Motion planning	difficult (many DOF)	more difficult (nonholonomic + DOF)
3. Motion control	difficult (inertial couplings)	more difficult (nonlinear, no smooth stabilizer)

Table 1.1: Comparison between fixed-base manipulators and wheeled mobile robots. The apparent simplicity of WMRs hides fundamental difficulties.

The real challenge, however, lies in *multi-body* mobile robots: mobile manipulators, tractor-trailer systems, and humanoids. These combine the planning and control difficulties of manipulators with the nonholonomic constraints and nonlinear dynamics of WMRs.

1.4 Autonomy

Definition 1.1 (Autonomy)**.** *A robot is* autonomous *when it can solve the three key problems (localization, planning, control) in* unstructured *environments, under* uncertain *and possibly* dynamic *operating conditions.*

Progress in autonomy is easy to overestimate. The DARPA Grand Challenge (2005) demonstrated autonomous driving in the desert, and the DARPA Robotics Challenge (2015) demonstrated legged locomotion and manipulation in semi-structured scenarios. Yet real autonomy — especially when manipulation is involved — is still far from being solved.

1.5 Perception

Autonomy rests on *perception*, i.e. sensing plus interpretation. Two categories are distinguished:

- **Proprioceptive**: perception of the robot itself (position, orientation, velocity). Sensors: encoders, IMUs, GPS.
- **Exteroceptive**: perception of the environment (obstacles, other robots, people). Sensors: rangefinders, cameras, tactile sensors.

Perception is essential in unstructured environments, where the state of the robot and of the world cannot be assumed to be known.

1.6 Robot Architectures

Three paradigmatic ways of organizing perception, planning, and control on a robot:

1.6.1 Deliberative Architecture

"*Think, then act.*" The robot builds a model of the world, plans on it, and then executes. The classical pipeline is:

`mission objectives` → `planning` → `path/trajectory` → `control`

with `perception` feeding `localization (SLAM)` and `planning`, and `control` sending actuator commands to the physical robot.

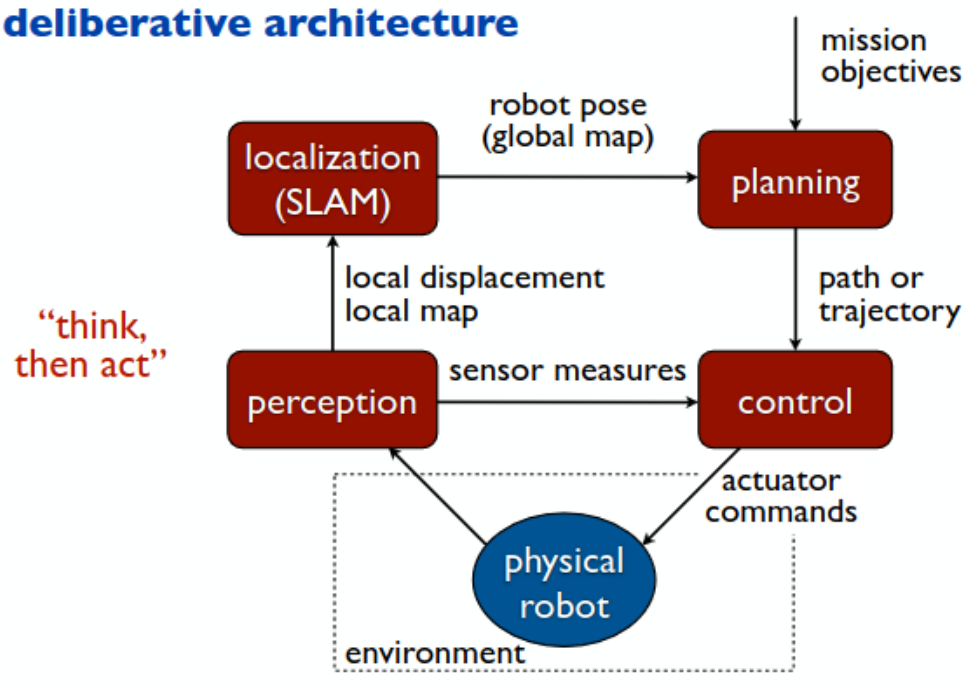


Figure 1.1: Deliberative architecture.

This is the reference architecture of the course.

1.6.2 Reactive Architecture

"Don't think, (re)act." No explicit model: direct mapping from sensor readings to actuator commands. Fast, but unable to handle non-trivial goals.

1.6.3 Hybrid Architectures

"Think and act concurrently." A set of elementary behaviors (e.g. *avoid obstacles*, *follow wall*, *detect goal*) is coordinated by a fusion block (e.g. vector summation). Deliberative and reactive layers coexist, each handling what it does best. A deliberative layer handles long-horizon reasoning (mission planning, global localization), while a reactive layer handles short-horizon reflexes (obstacle avoidance, emergency stops). The two run in parallel and exchange information through a shared representation of the world.

1.6.4 Behavior-Based Architecture

"Think the way you act." Rather than decomposing the problem into functional modules (perception, planning, control), a behavior-based architecture decomposes it into *behaviors*: elementary sensor-to-actuator mappings, each responsible for a specific task. Each behavior reacts to the current sensor readings and produces a *proposal* for the actuators. A **coordination/fusion** block combines these proposals into the final actuator command — for example, by vector summation, priority arbitration, or subsumption.

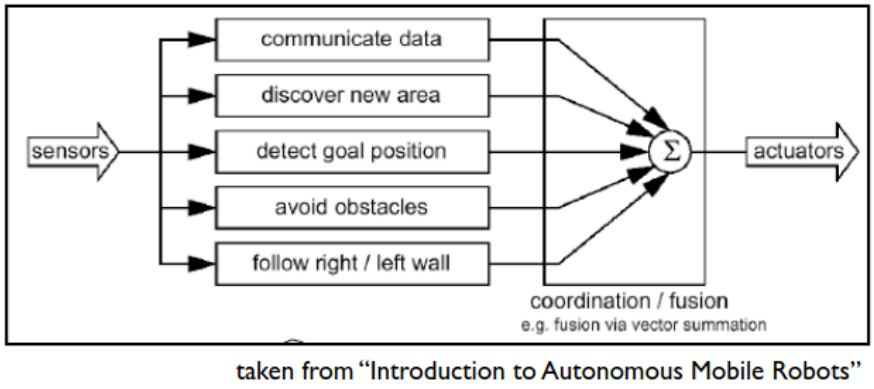


Figure 1.2: Behavior-based architecture: elementary behaviors are coordinated by a fusion block (e.g. vector summation) before reaching the actuators. Adapted from [Cho+05].

1.7 Course Approach

The course adopts a *model-based* approach, for the following reasons:

- Robots cannot escape physics.
- Models encode structured prior knowledge about the robot and the world — knowledge that does not need to be learned again.
- Machine learning can capture effects that are unknown a priori, difficult to model, or accessible only through data.
- Roboticists must be able to reason rigorously about autonomous physical systems, critically evaluate learned solutions, and combine the two.

  Use models for what we know, data for what we don't know, and feedback for what is still missing.

Chapter 2

Configuration Space

The configuration space is the unifying abstraction of robot motion: once it is in place, planning, control, and localization all become statements about the same underlying object. This chapter introduces the concept, its structure, and its role in motion planning.

2.1 Motivation: Why Configuration Space?

The central problem of mobile robotics is the following: given a robot in an environment with obstacles, move it from an initial configuration to a final one without collisions. Stated directly in the workspace, this problem is hard: the robot has an extent, the obstacles have arbitrary shapes, and checking whether a given placement is collision-free requires geometric reasoning.

The *configuration space* (C-space) makes the problem tractable. The idea is to replace the motion of an extended body among obstacles with the motion of a *point* in an abstract space, where obstacles have been "inflated" to account for the robot's shape: the robot is represented as a point, and the obstacles are represented as regions of that space that the point cannot enter. This reduction turns a complex geometric problem into a path-search problem in a topological space, for which a large body of algorithms exists [Cho+05; Sic+25].

2.2 Modeling a Robot

A robot is a continuous body, i.e. an infinite set of points. Describing it point by point is impossible. The key idea is to represent it with a *finite* set of parameters that completely determines the position of every point of the robot: the configuration. We model a robot as a collection of n_b rigid bodies moving in a workspace W. The workspace is the set of physical points in which the robot can move.

2.2.1 Configuration and Degrees of Freedom

Definition 2.1 (Configuration)**.** *A* configuration q *is a complete specification of the position of every point of the robot. It is represented as a vector* $q = [q_1, \dots, q_n]^\top$*, where* n *is the number of* degrees of freedom *(DOF) of the robot.*

Each q_i is a generalized coordinate, either Cartesian or angular. The Cartesian coordinates identify the position of points of the robot (e.g. the center of mass of a rigid body), and take values in $\mathbb{R}$. The angular coordinates identify the orientation of the rigid bodies, and take values in $\mathcal{S}^1 \cong SO(2)$ (planar rotations) or in $SO(3)$ (spatial rotations).

Thus, a *configuration* is a minimal set of parameters that identifies the posture of the robot in W: if the parameters are known, the position of every point of the robot is known.

Special Orthogonal Group SO(n)

The Special Orthogonal group $SO(n)$ is the group of rotations in $\mathbb{R}^n$ that preserve orientation and the Euclidean structure. It is characterized by orthogonal matrices with determinant equal to one, representing transformations that maintain distances and angles.
All the matrices $R \in SO(n)$ satisfy the following properties:

- $R^\top R = I$, where $R^\top$ is the transpose of R and I is the identity matrix;
- $\det(R) = 1$, ensuring that the transformation preserves orientation;
- The set of all such matrices forms a group under matrix multiplication.

2.3 The Configuration Space and the Workspace

Definition 2.2 (Configuration Space)**.** *The* configuration space C *is the set of all possible configurations of the robot. It is an* n*-dimensional* manifold*, i.e. a space that locally looks like* $\mathbb{R}^n$*, but that in general is* not $\mathbb{R}^n$ *globally. Typically it is a product of Euclidean factors and circular (rotational) factors: for instance,* $C = \mathbb{R}^2 \times \mathcal{S}^1$ *for a planar robot with orientation, where* $\mathcal{S}^1$ *denotes the unit circle.*

For a collection of n_b *free* rigid bodies moving in the plane, $\dim(C) = 3n_b$ (two translations and one rotation per body). In three dimensions, $\dim(C) = 6n_b$ (three translations and three rotations). Joints and other constraints between the bodies reduce these numbers: for a manipulator with n one-DOF joints, In all cases,

$$\dim(C) = \text{DOF of the robot},$$

the minimum number of independent parameters required to uniquely specify its state.

When the configuration space of a free rigid body is considered, it has the structure of the *Special Euclidean Group*: $SE(2) \cong \mathbb{R}^2 \times \mathcal{S}^1$ for a body in the plane, and $SE(3) = \mathbb{R}^3 \times SO(3)$ for a body in space.

Special Euclidean Group SE(n)

The Special Euclidean group $SE(n)$ describes the orientation-preserving isometries of the Euclidean space $\mathbb{R}^n$, where $n \in \mathbb{N}$ is an integer. It represents rigid transformations (combinations of rotations and translations) that preserve both distances between points and the orientation of the space.
This group can be represented using homogeneous transformation matrices as:

$$SE(n) = \left\{ X = \begin{pmatrix} R & p \\ 0_{1\times n} & 1 \end{pmatrix} \in \mathbb{R}^{(n+1)\times(n+1)} \;\middle|\; R^\top R = I_n,\ \det(R) = 1,\ p \in \mathbb{R}^n \right\}$$

The matrix components are defined as follows:

- **Rotation matrix (R):** An element of the Special Orthogonal group $SO(n)$, characterized by $R^\top R = I_n$ and $\det(R) = 1$, which handles changes in orientation without deformation or reflection.
- **Translation vector (p):** A vector $p \in \mathbb{R}^n$ representing the rigid displacement in space.
- **Bottom row ($0_{1\times n}$ and 1):** A padding row that establishes homogeneous coordinates, allowing the composition of multiple rigid transformations through simple matrix multiplication.

As a manifold, $SE(n) \cong \mathbb{R}^n \times SO(n)$, with dimension $n(n+1)/2$: 3 for $SE(2)$, 6 for $SE(3)$.

2.3.1 Workspace vs Configuration Space

To recap:

- The **workspace** W is the physical space in which the robot moves (e.g. $\mathbb{R}^2$ or $\mathbb{R}^3$), expressed in world coordinates.
- The **configuration space** C is the abstract space of all configurations the robot can assume: the manifold of all its possible postures.

In general $\dim(C) \neq \dim(W)$: the dimension of C depends on the kinematic structure of the robot, not on the space it lives in. A planar robot that can both translate and rotate has a 3-dimensional C-space, even though it lives in a 2-dimensional workspace. A free rigid body in $\mathbb{R}^3$ has a 6-dimensional C-space, while its workspace is 3-dimensional.

2.4 Examples

2.4.1 Point Robot

If the robot is a point translating in $\mathbb{R}^2$, then $q = (x, y)$ and $C = \mathbb{R}^2$. The C-space coincides with the workspace, and the C-obstacles are the original obstacles.

2.4.2 Disk Robot

A disk-shaped robot of radius r translating in $\mathbb{R}^2$ has $q = (x, y)$, where (x, y) is the center of the disk. Again $C = \mathbb{R}^2$, but each obstacle is *inflated* by the robot's radius: a point obstacle becomes a disk of radius r, and a polygonal obstacle is expanded by the *Minkowski sum* with the robot's disk. After this inflation, the disk robot can be treated as a point in C.

2.4.3 Robot with Orientation: $\mathbb{R}^2 \times \mathcal{S}^1$

If the robot is a rigid body that can both translate and rotate in the plane (e.g. a polygon, for which orientation matters), then $q = (x, y, \theta)$ and $C = \mathbb{R}^2 \times \mathcal{S}^1$, where $\mathcal{S}^1$ is the unit circle (the angle θ is periodic). This space has a non-trivial topology: it is *not* $\mathbb{R}^3$, because θ and $\theta + 2\pi$ denote the same configuration. As a manifold it is $SE(2)$. The intuitive picture is a *thick cylinder*: two linear dimensions (x, y) and one circular dimension (θ), i.e. a circle of orientations attached to every point of the plane (the 2D analogue is the cylinder $\mathbb{R} \times \mathcal{S}^1$).

2.4.4 Planar 2-Link Manipulator: $\mathcal{T}^2$

For a planar manipulator with two revolute joints, $q = (\theta_1, \theta_2)$ and $C = \mathcal{S}^1 \times \mathcal{S}^1 = \mathcal{T}^2$, the *2-dimensional torus*. Each joint is a circle, and the product of two circles is a torus. This is a classic example of how the topology of C can differ radically from that of $\mathbb{R}^n$.

2.4.5 Rigid Body in 3D: $SE(3)$

A free rigid body in $\mathbb{R}^3$ has 6 DOF: three translational and three rotational. Its C-space is $C = \mathbb{R}^3 \times SO(3) = SE(3)$, the *special Euclidean group* of dimension 6. The rotational part $SO(3)$ is not $\mathbb{R}^3$: it is a compact 3-dimensional manifold.

2.5 Topology of Configuration Space

The configuration space C of a robot is not always a Euclidean space. Its *topology* — the way it is glued together globally — can differ substantially from that of $\mathbb{R}^n$, and this has direct consequences on the design of planning and control algorithms. The canonical example in this course is the planar 2R manipulator, which we use to answer three questions in order: what is C for this robot, why does the naive picture of it as a plane fail, and what is the right space to work in.

2.5.1 The 2R Planar Manipulator

Consider a planar manipulator with two revolute joints. Its configuration is $q = (q_1, q_2)^\top$, where q_1 is the angle of the first link with respect to the base frame and q_2 the angle of the second link with respect to the first (fig. 2.1, left). Each joint angle is a planar rotation, i.e. an element of $SO(2) \cong \mathcal{S}^1$, the circle. The configuration space is therefore the product of two circles,

$$C = SO(2) \times SO(2) \cong \mathcal{S}^1 \times \mathcal{S}^1 = \mathcal{T}^2,$$

the *2-dimensional torus*.

The most immediate way to draw C is to plot every configuration as a point of the plane $(q_1, q_2) \in \mathbb{R}^2$ (fig. 2.1, right): each point is a posture of the arm. This picture is, however, misleading.

2.5.2 Non-Injectivity of the Parameterization

Angles are periodic with period 2π: rotating a joint by a full turn leaves the arm exactly where it was. The points $(\bar{q}_1, \bar{q}_2)$ and $(\bar{q}_1 + 2\pi, \bar{q}_2)$ of fig. 2.1 are therefore *different points describing the same posture*. Formally, the map

$$\Phi : \mathbb{R}^2 \to \mathcal{T}^2, \qquad \Phi(q_1, q_2) = (q_1 \bmod 2\pi,\ q_2 \bmod 2\pi),$$

is *surjective but not injective*: two points have the same image if and only if they differ by integer multiples of 2π in each coordinate,

$$(q_1, q_2) \sim (q_1 + 2\pi k_1,\ q_2 + 2\pi k_2), \qquad k_1, k_2 \in \mathbb{Z},$$

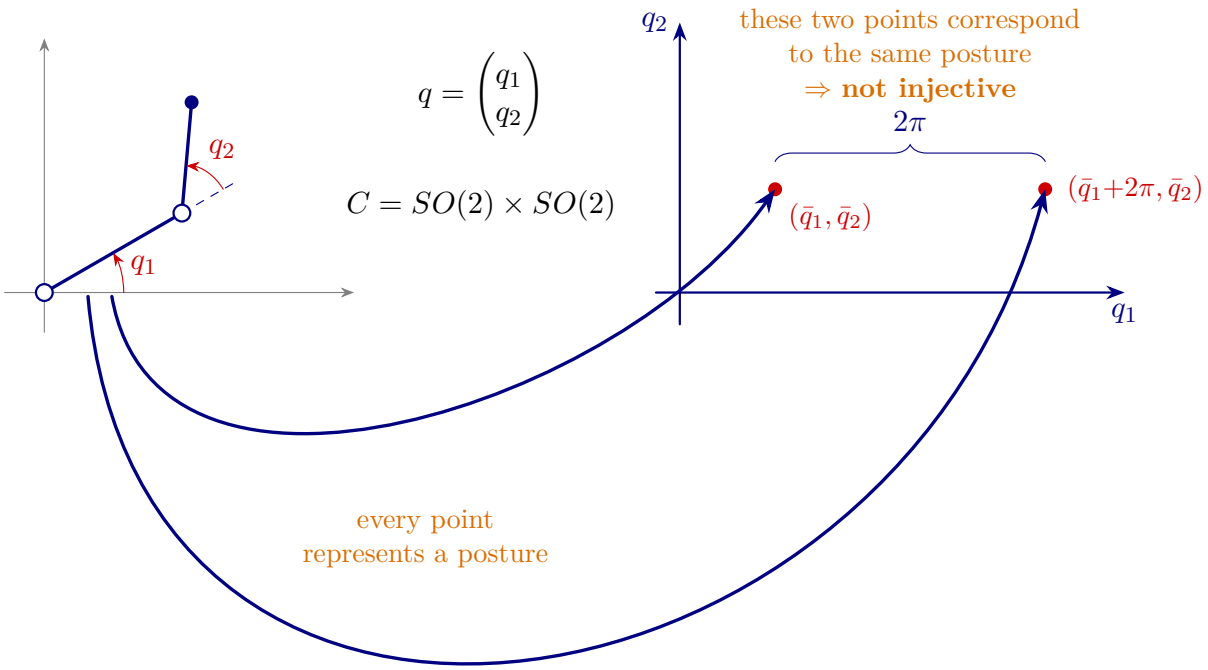


Figure 2.1: The planar 2R arm and its (q_1, q_2) plane. Every point of the plane is a posture, but $(\bar{q}_1, \bar{q}_2)$ and $(\bar{q}_1 + 2\pi, \bar{q}_2)$ are different points describing the *same* posture: the parameterization is not injective.

so every posture has infinitely many representatives in the plane. Restricting to the closed square $[0, 2\pi] \times [0, 2\pi]$ is not enough: opposite edges are still pairwise identified, $(q_1, 0) \sim (q_1, 2\pi)$ and $(0, q_2) \sim (2\pi, q_2)$, and the four corners all represent the same posture.

2.5.3 Restoring Injectivity: the Fundamental Domain

To remove the redundancy it is enough to keep one representative per class. The standard choice is the half-open square

$$[0, 2\pi) \times [0, 2\pi) \subset \mathbb{R}^2,$$

called the *fundamental domain*: the left and bottom edges belong to C, while the top and right edges do not (dashed in fig. 2.2, left). With this choice Φ becomes a *bijection*: for each configuration in the square there is one and only one posture, and every posture appears exactly once. This is why the fundamental domain is usually written as a half-open square.

2.5.4 The Remaining Problem: Closeness Is Not Preserved

Injectivity is not enough. A good representation of C must also respect its *topology*: postures that are close must be represented by points that are close, and vice versa. The square fails this test. Take a small $\varepsilon > 0$ and consider $q_\varepsilon = (\varepsilon, q_2)$ and $q_{2\pi-\varepsilon} = (2\pi - \varepsilon, q_2)$ (fig. 2.2). In the square they are almost as far apart as possible, at distance $2\pi - 2\varepsilon$; but the two arms differ only by a rotation of 2ε of the first joint, and as $\varepsilon \to 0$ they coincide (fig. 2.2, right). Equivalently, a continuous motion of the arm across $q_1 = 0$ (or $q_2 = 0$) appears in the square as a jump from one edge to the opposite one.

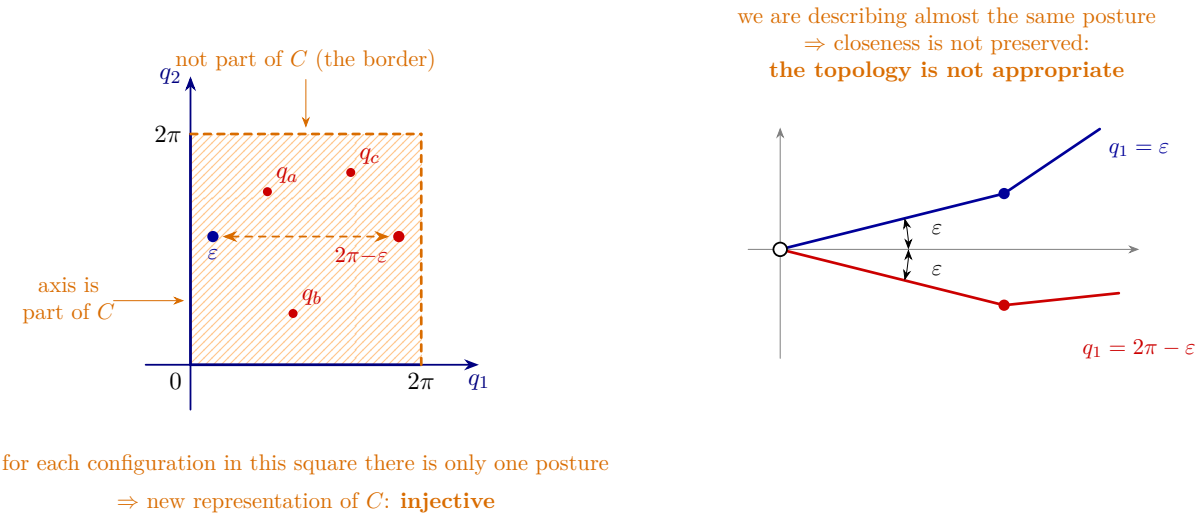


Figure 2.2: Left: the half-open square $[0, 2\pi) \times [0, 2\pi)$ (dashed sides excluded) gives exactly one point per posture, so the representation is injective. Right: two points at opposite sides of the square are far apart in the plane but describe almost the same posture, so the square does not respect the topology of C.

In mathematical terms, Φ restricted to the half-open square is a continuous bijection onto $\mathcal{T}^2$ whose inverse is discontinuous along the edges, so it is not a homeomorphism: the fundamental domain describes the same set of postures, but with the wrong topology.

2.5.5 Gluing the Square: Why C Is a Torus

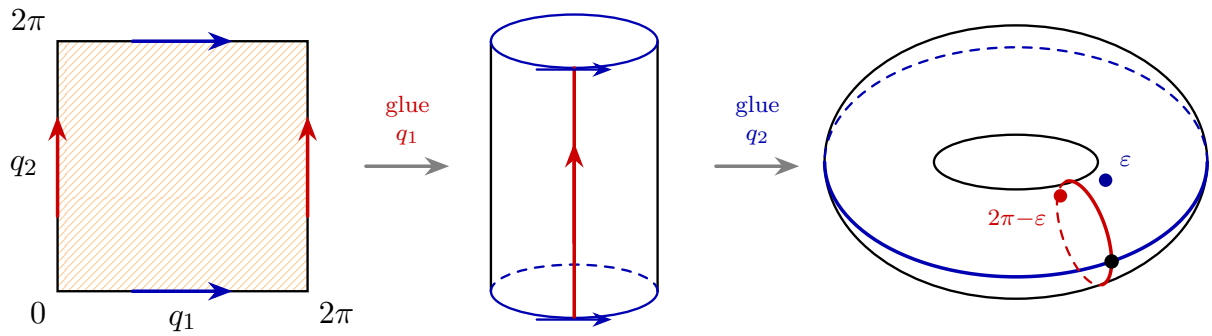
The fix is to put back what the fundamental domain removed, namely the identifications $q_i = 0 \equiv 2\pi$ on the boundary, so that points on opposite edges become neighbours again. This can be done in two steps (fig. 2.3). Gluing the vertical edges $q_1 = 0 \equiv 2\pi$ turns the square into a cylinder; gluing the two circular boundaries $q_2 = 0 \equiv 2\pi$ of the cylinder closes it into a torus. The points q_ε and $q_{2\pi-\varepsilon}$ now lie on the two sides of the seam and are indeed close to each other: the torus respects the topology of C.

That the result is always a torus is not a coincidence but a consequence of the product structure: each joint contributes a circle, and the product of two circles is $\mathcal{T}^2$ (for n revolute joints, $\mathcal{T}^n$). The torus is also a *manifold*: around every point, seam included, a small neighbourhood looks like a disc of $\mathbb{R}^2$. Angular coordinates (q_1, q_2) are thus perfectly good *locally*, and the only obstruction is to use them *globally* with a single chart, which is what the square attempts to do. This is typical: configuration spaces are usually manifolds described by angular coordinates.

2.5.6 Why Topology Matters

The toroidal topology of C is not a mathematical curiosity: it has practical consequences.

- **Paths and interpolation.** A straight line in the square $[0, 2\pi)^2$ between q_ε and $q_{2\pi-\varepsilon}$ sweeps almost the whole range of the first joint, whereas on the torus the shortest path is the short arc across the seam, of length 2ε. Planners must wrap coordinates modulo 2π, not clamp them.
- **Distance metrics.** The Euclidean distance $\|q - q'\|$ is not the natural metric on $\mathcal{S}^1$: the correct angular distance is $\min(|q - q'|,\ 2\pi - |q - q'|)$.
- **Continuous parameterizations.** There is no globally continuous, single-valued parameterization of $\mathcal{S}^1$ by a real number: no interval of $\mathbb{R}$ is homeomorphic to the circle, so every



(a) square (b) cylinder (c) torus $\mathcal{T}^2$

Figure 2.3: Gluing the opposite sides of the square: identifying $q_1 = 0 \equiv 2\pi$ (red) gives a cylinder, identifying $q_2 = 0 \equiv 2\pi$ (blue) closes it into a torus $\mathcal{T}^2 = \mathcal{S}^1 \times \mathcal{S}^1$. The points $q_1 = \varepsilon$ and $q_1 = 2\pi - \varepsilon$, far apart in the square, are now neighbours on either side of the seam: the torus respects the topology of C. Configuration spaces are in general manifolds (angular coordinates).

real representation $q \in \mathbb{R}$ has a discontinuity at the wrap-around point. Controllers must account for this.

These issues generalize to any non-Euclidean configuration space: $\mathcal{T}^n$ for manipulators with revolute joints, $SE(3)$ for free-flying bodies, $\mathbb{R}^2 \times \mathcal{S}^1$ for wheeled mobile robots with orientation.

Torus vs. Sphere

The torus $\mathcal{T}^2$ and the sphere $\mathcal{S}^2$ are both 2-dimensional compact manifolds, but they are *not* homeomorphic: the torus has genus 1 (one hole), the sphere has genus 0. On the torus, a loop that wraps around the hole cannot be continuously contracted to a point; on the sphere, every loop can. This distinction is what makes the topology of C matter for planning.

2.6 Distance in the Configuration Space

Planning, control, and localization all require a notion of *distance* between configurations: how far apart are two postures of the robot? In a Euclidean configuration space the answer is the Euclidean norm. On a general manifold the answer is more subtle, and choosing the right metric has practical consequences.

Definition 2.3 (Distance)**.** *Given a configuration space C, a* distance *is a function $d : C \times C \to \mathbb{R}$ such that, for every $q_a, q_b, q_c \in C$:*

$$d(q_a, q_b) \geq 0, \tag{2.1}$$
$$d(q_a, q_b) = 0 \iff q_a = q_b, \tag{2.2}$$
$$d(q_a, q_b) = d(q_b, q_a), \tag{2.3}$$
$$d(q_a, q_b) \leq d(q_a, q_c) + d(q_c, q_b). \tag{2.4}$$

2.6.1 Geodesic Distance

In $\mathbb{R}^n$, the distance between two vectors $\mathbf{u}, \mathbf{v}$ is $\|\mathbf{u} - \mathbf{v}\|$. On a general manifold, the distance between two points is the length of the *shortest path* connecting them.

Definition 2.4 (Geodesic)**.** *For a general manifold, the shortest path between two points is called the* ***geodesic****, and the distance between the points is the length of their geodesic.*

For example, on a sphere (a 2-manifold), the geodesic between two points lies on the *great circle* passing through them. In general, computing the geodesic on an arbitrary manifold is hard, and for a robot configuration space it is rarely the right tool.

2.6.2 The Displacement Metric

In robotics we want to measure the distance between two *postures*, i.e. between two regions of the workspace occupied by the robot. This can be done directly in the workspace, without computing geodesics on C. Two functions are useful:

- $B(q)$: the region of the workspace occupied by the robot at configuration q, i.e. a set of points in $\mathbb{R}^3$.
- $\mathbf{p}(q)$: the position of a specific point of the robot (e.g. its center of mass) at configuration q.

A natural metric is the *maximum displacement* of any point of the robot between the two configurations:

$$d(q_a, q_b) = \max_{\mathbf{p} \in \text{robot}} \|\mathbf{p}(q_a) - \mathbf{p}(q_b)\|. \tag{2.5}$$

This is called the **displacement metric**. It relies on the assumption that if the region occupied by the robot is similar in two postures, then the distance between the corresponding configurations is small:

$$d(q_a, q_b) \simeq 0 \implies B(q_a) \simeq B(q_b).$$

Computing eq. (2.5) exactly requires checking every point of the robot, which is an uncountably infinite set. If the links are polyhedral, the problem becomes finite: the maximum displacement is attained at a vertex, so it is enough to check the vertices of the bodies. For general shapes, a more practical approach is to use a finite set of **control points**.

2.6.3 Control Points

Let

$$P = \{\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_c\}$$

be a set of c points of the robot, each depending on the configuration q. The distance between two postures is defined as

$$d(q_a, q_b) = \max_{1 \le i \le c} \|\mathbf{p}_i(q_a) - \mathbf{p}_i(q_b)\|. \tag{2.6}$$

Not every set of points is a valid choice. For instance, the base of a manipulator does not move with the configuration, so it carries no information. A set P is a set of **proper control points** if the function d defined in eq. (2.6) satisfies all the properties of a distance in definition 2.3.

Local Approximation

Since a smooth manifold is locally homeomorphic to $\mathbb{R}^n$, for sufficiently close configurations the geodesic distance on the manifold can be approximated by the Euclidean distance in the local chart.

Prismatic Joints

The configuration space of a robot with *only* prismatic joints is Euclidean, because each joint contributes a linear coordinate. The distance in C is then the ordinary Euclidean distance.

2.7 C-Obstacles and Free Space

Once C is defined, obstacles from the workspace must be mapped into it.

Definition 2.5 (C-Obstacle)**.** *Given an obstacle $O_i \subset W$, the* C-obstacle CO_i *is the set of configurations in which the robot collides with O_i:*

$$CO_i = \{q \in C : B(q) \cap O_i \neq \emptyset\},$$

where $B(q)$ is the volume occupied by the robot at configuration q.

The union of all C-obstacles is the *C-obstacle region*:

$$CO = \bigcup_i CO_i.$$

Definition 2.6 (Free Configuration Space)**.** *The* free configuration space *is the complement of CO in C:*

$$C_{free} = C \setminus CO = \{q \in C : B(q) \cap \textstyle\bigcup_i O_i = \emptyset\}.$$

Planning a safe motion is equivalent to finding a path in C_{free} connecting the start configuration q_s to the goal configuration q_g.

The point-robot trick. To plan for a robot of finite size as if it were a point, the workspace obstacles are inflated by the *Minkowski sum* with the robot's geometry: for a robot A that only translates, with its reference point at the origin, $CO_i = O_i \oplus (-A)$, i.e. the obstacle is grown by the robot reflected through the reference point (for a symmetric robot such as a disk, simply by the robot itself). After this transformation, planning for the extended robot reduces to planning for a point in C_{free}. When the robot can also rotate, the inflation depends on the orientation: one inflated obstacle is obtained for each θ, and the C-obstacle lives in the higher-dimensional space $\mathbb{R}^2 \times \mathcal{S}^1$. This is the canonical formulation of the motion planning problem.

Path vs. trajectory. A *path* is a purely geometric sequence of configurations, with no timing information. A *trajectory* is a path endowed with a time law $q(t)$. In this chapter, and in motion planning in general, the object of interest is the path. Timing becomes relevant only when the dynamics and the control layer are brought into play.

2.8 Relevance to Motion Planning

The configuration space reformulates the motion planning problem in a clean and general way:

> Given C, a start configuration $q_s \in C_{\text{free}}$, and a goal configuration $q_g \in C_{\text{free}}$, find a continuous path $\tau : [0, 1] \to C_{\text{free}}$ such that $\tau(0) = q_s$ and $\tau(1) = q_g$.

All the algorithms studied later in the course — roadmaps, cell decompositions, probabilistic planners, potential fields — are methods for solving this problem in C_{free}. The differences between them lie in how they represent C_{free} and how they search it.

A subtle point: the *topology* of C matters. Searching for a path in $\mathbb{R}^n$ is different from searching in $\mathcal{S}^1$, $\mathcal{T}^n$, or $SE(3)$. Planners must respect the topology of the space in which they operate.

The kinematic constraints that actually limit the motion of a wheeled mobile robot are the subject of chapter 3, where they are derived from the pure rolling condition and used to build the kinematic models.

Chapter 3

Wheeled Mobile Robots: Mechanics

This chapter introduces the mechanical structure of wheeled mobile robots. It will focus on what kinds of wheels exist, how they can be assembled into kinematic structures, and what constraints are imposed on the motion of the robot in various situations.

3.1 Sources of Locomotion

The main sources of movement for a mobile robot are:

- **Wheels**: wheeled robots usually consist of a single rigid body, with the wheels in contact with the ground.

- **Feet**: legged robots, consisting of several rigid bodies articulated through joints.

In this course we focus on wheeled mobile robots (WMRs), which are the simplest and most widely deployed class of mobile robots.

3.2 Static Balance

The wheels are in contact with the ground, and each wheel defines a point of contact. The **support polygon** is the convex hull of all the contact points.

Definition 3.1 (Static Balance)**.** *A wheeled robot is in* ***static balance*** *if the projection on the ground of its center of mass falls inside the support polygon.*

A robot must be balanced in order not to fall. The convex hull of two points is a line segment, hence a robot with only two wheels cannot be easily balanced in a static fashion: it needs at least a third contact point (typically a caster wheel). In the same way, a vehicle with a single wheel (the unicycle) has a support polygon that degenerates to a point, and a bicycle has a support polygon that degenerates to a line segment: neither is statically balanced.

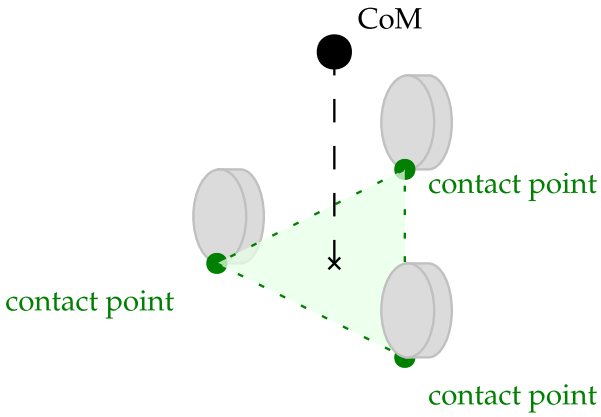


Figure 3.1: Support polygon and static balance.

3.3 Types of Wheels

There are three main types of wheels:

- **Fixed wheels**: they have a fixed orientation with respect to the robot body. They can roll but cannot steer about their vertical axis. They are usually actuated; if passive, they are used for balance and offer a contact point.
- **Steerable wheels**: they can steer and are used for orientation, hence they are typically active (e.g. the front wheels of a car).
- **Caster wheels**: they have a variable orientation but are typically not actuated. Their axis of orientation is not aligned with the wheel axis, so they align automatically with the direction of motion. They are kinematically "invisible" but offer a contact point for balance.

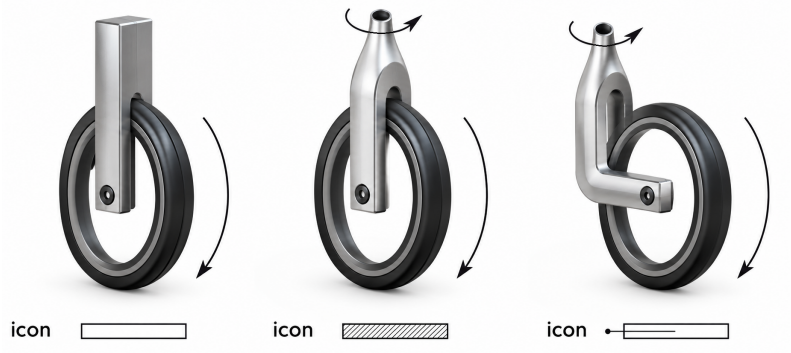


Figure 3.2: Fixed, steerable, and caster wheels.

3.4 Kinematic Structures

3.4.1 Differential Drive

The simplest wheeled robot is the **differential drive**: two aligned fixed wheels, independently actuated, plus one caster wheel for balance. The two actuated wheels are driven independently, which allows the robot to rotate in place by making them turn in opposite directions.

3.4.2 Synchro-Drive

A **synchro-drive** robot has three steerable wheels and two actuators: one controls simultaneously the orientation of all the wheels, and the other rolls them together. The base over the wheels never changes orientation. This configuration is preferable when the body to be moved is particularly heavy.

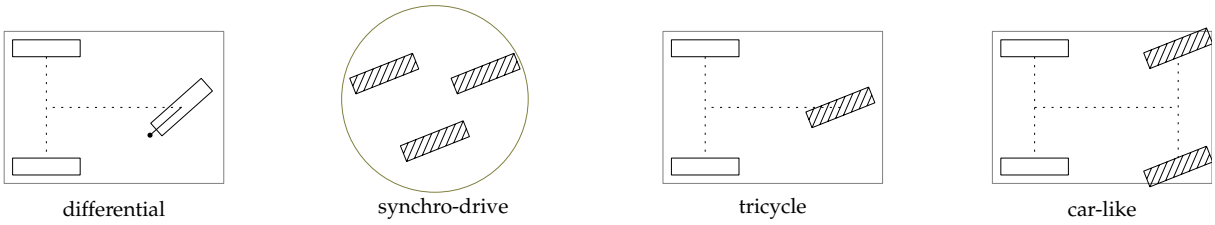


Figure 3.3: Differential drive (left) and synchro-drive (right).

3.4.3 Tricycle and Car-Like

The **tricycle** has two fixed wheels at the back and one actuated steerable wheel at the front. The **car-like** robot is similar, but with two steerable wheels at the front.

Both are subject to *nonholonomic* constraints: the pure rolling condition on each wheel forbids sideways motion, but does not prevent the robot from reaching any configuration in the plane.

3.5 The Differential

Consider a car-like robot taking a curve. In the same time interval, the outer wheel travels a longer distance than the inner wheel, so its rotation speed must be higher. If the two front wheels were rigidly connected to a single motor, they would be forced to spin at the same speed, causing tire slip and mechanical strain.

To avoid this, a device called **differential** is used.

Definition 3.2 (Differential)**.** *A mechanical differential is a gear assembly (typically bevel or planetary gears) connected to a single drive motor. It splits the power to the two wheels on the same axle while allowing them to rotate at different speeds. When the robot turns, the outer wheel travels a longer arc than the inner one; the differential lets each wheel spin at its natural speed, preventing tire slip, scrubbing, and mechanical strain on the drivetrain.*

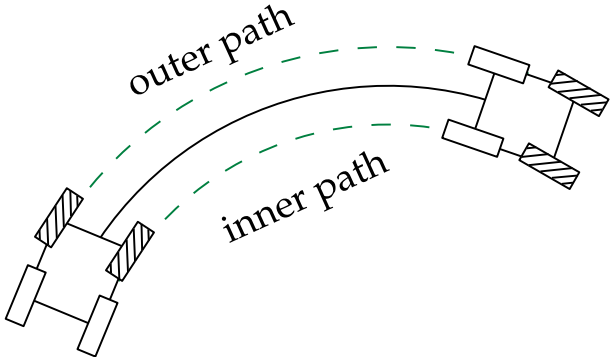


Figure 3.4: Path of a car-like robot turning: the outer wheel travels a longer distance than the inner one.

3.6 Constraints

The presence of wheels in contact with the ground introduces *constraints* on the motion of the robot. Two classes matter for this course: geometric constraints, which act on the configuration, and kinematic constraints, which act on the velocities.

3.6.1 Geometric Constraints

We focus on equality constraints that are time-invariant. A geometric constraint has the form

$$h(q) = 0, \tag{3.1}$$

and depends only on the configuration. In general we have $k < n$ independent constraints,

$$h_1(q) = 0, \tag{3.2}$$

$$\vdots \tag{3.3}$$

$$h_k(q) = 0. \tag{3.4}$$

If the configuration space is an n-manifold, k independent geometric constraints reduce the accessible configurations to an $(n-k)$-manifold embedded in C, called a **sub-manifold**.

For example, if $C = \mathbb{R}^2$ (a 2-manifold), one constraint restricts the configurations to a curve in the plane. If $C = \mathbb{R}^3$, one constraint restricts them to a surface.

Solving for a Coordinate

Instead of using the full vector q, one can solve $h(q) = 0$ for one coordinate, e.g. $q_i = \tilde{h}(q_1, \ldots, q_{i-1}, q_{i+1}, \ldots, q_n)$. But this choice is not always good: for the constraint $q_1^2 + q_2^2 = 1$, solving for q_2 gives $q_2 = \pm\sqrt{1 - q_1^2}$, which is not single-valued. For a circle, it is better to use the angle $\alpha \in \mathcal{S}^1$ as the single coordinate, with $q_1 = \cos\alpha$ and $q_2 = \sin\alpha$.

3.6.2 Kinematic Constraints

A kinematic constraint is local: it prevents the configuration from moving in certain directions, but does not necessarily restrict the accessible configurations. A general kinematic constraint has the form

$$a(q, \dot{q}) = 0, \tag{3.5}$$

and depends on the state and its velocity. All the relevant kinematic constraints in robotics are in **Pfaffian form**:

$$a^\top(q)\dot{q} = 0, \tag{3.6}$$

with $a(q)$ an n-dimensional vector depending on the configuration. The Pfaffian form expresses an orthogonality condition: at each configuration, the velocity must be orthogonal to $a(q)$.

Multiple kinematic constraints can be arranged in matrix form:

$$\begin{pmatrix} a_1^\top(q) \\ a_2^\top(q) \\ \vdots \\ a_k^\top(q) \end{pmatrix} \dot{q} = 0 \implies A(q)\dot{q} = 0 \iff \dot{q} \in \mathcal{N}(A(q)), \tag{3.7}$$

where $A(q)$ is a $k \times n$ matrix. The admissible velocities are the elements of the null space of $A(q)$, a linear subspace of $\mathbb{R}^n$.

Geometric Implies Kinematic

Any geometric constraint $h(q) = 0$ implies a kinematic constraint. Differentiating with respect to time:

$$\dot{h}(q) = \frac{\partial h}{\partial q}\dot{q} = (\nabla_q h)^\top \dot{q} = 0.$$

This is exactly a Pfaffian constraint with $a(q) = \nabla_q h$. The gradient is orthogonal to the sub-manifold defined by the geometric constraint, so the kinematic constraint prevents the robot from leaving the sub-manifold.

The converse is not generally true: a kinematic constraint $a^\top(q)\dot{q} = 0$ implies a geometric constraint only if the constraint is *integrable*, i.e. only if there exist a function $h(q)$ and a nonvanishing function $\lambda(q)$ such that

$$\nabla_q h = \lambda(q)\, a(q). \tag{3.8}$$

(The factor λ is needed because multiplying a constraint by a nonzero function does not change the set of admissible velocities; in the simplest case $\lambda = 1$.)

3.6.3 Holonomic vs Nonholonomic Constraints

Definition 3.3 (Holonomic Constraint)**.** *An integrable kinematic constraint is called* ***holonomic****. It defines a geometric constraint, and therefore reduces the dimension of the accessible configuration space.*

Definition 3.4 (Nonholonomic Constraint)**.** *A kinematic constraint that is not integrable is called* ***nonholonomic****. It restricts the set of admissible velocities at each configuration, without being equivalent to a geometric constraint. Depending on the degree of integrability, the accessible configuration space is either the whole C (*completely nonholonomic *constraints) or a sub-manifold of dimension strictly larger than the number of independent admissible velocity directions (*partially integrable *constraints, see section 3.7.1).*

Example: integrable constraint. Consider the kinematic constraint for a 2-dimensional robot:

$$q_1\dot{q}_1 + q_2\dot{q}_2 = 0.$$

This is integrable: the resulting geometric constraint is $q_1^2 + q_2^2 = c$, with $c > 0$. So it is holonomic.

Example: nonholonomic constraint. The simplest nonholonomic system is the **rolling disk**. Its configuration is $q = (x, y, \theta)^\top$, with θ the orientation with respect to the horizontal axis. In pure rolling (no slippage), the disk can only move in the direction it points to, not laterally. The constraint is

$$\sin\theta\,\dot{x} - \cos\theta\,\dot{y} = 0, \tag{3.9}$$

or in vector form

$$\begin{pmatrix}\sin\theta & -\cos\theta & 0\end{pmatrix}\begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{pmatrix} = 0.$$

The vector $n^\top = (\sin\theta, -\cos\theta)$ is normal to the direction of motion. Any velocity with a component along n is not allowed. The line through the contact point with direction n is called the **zero-motion line**: the velocity of the contact point has zero component along it.

This constraint does not restrict the reachable configurations: every configuration can still be reached. To see this, construct an explicit trajectory (a **universal maneuver**) between any two configurations q_s and q_g:

1. Rotate in place from θ_s to the angle θ^* such that the vector $(x_g - x_s, y_g - y_s)$ is aligned with the sagittal axis.
2. Move along that direction until (x_g, y_g) is reached.
3. Rotate in place from θ^* to θ_g.

Each step respects the rolling constraint. So the rolling disk is nonholonomic but controllable.

Holonomic vs Nonholonomic: The Takeaway

- **Holonomic** (integrable): the accessible configuration space is reduced to a sub-manifold. The robot is confined to it, hence the system is *not* controllable.
- **Completely nonholonomic**: the accessible configuration space is the whole C. The robot can reach any configuration, but only along admissible paths: the system is controllable.
- **Partially integrable** (still nonholonomic): the accessible configuration space is a sub-manifold, but larger than the one a holonomic constraint would give. The system is not controllable.

Controllability of the kinematic model is therefore equivalent to the constraints being completely nonholonomic. The precise test is given in section 3.7.1.

3.7 Kinematic Models

The kinematic constraints are written in Pfaffian form $A(q)\dot{q} = 0$, with $A(q)$ a $k \times n$ matrix. Admissible velocities belong to the null space of $A(q)$. We want to find a basis for this null space:

$$\mathcal{N}(A(q)) = \text{span}\{g_1(q), g_2(q), \ldots, g_m(q)\},$$

where $m := n - k$. Each $g_i(q)$ is a vector field on C with values in $\mathbb{R}^n$, and every admissible velocity is a linear combination of the basis vectors:

$$\dot{q} = \sum_{i=1}^{m} u_i \, g_i(q). \tag{3.10}$$

In compact form, introducing the m-dimensional vector $u = (u_1, \ldots, u_m)^\top$:

$$\dot{q} = \underbrace{\begin{pmatrix} | & | & & | \\ g_1(q) & g_2(q) & \cdots & g_m(q) \\ | & | & & | \end{pmatrix}}_{G(q)} \underbrace{\begin{pmatrix} u_1 \\ \vdots \\ u_m \end{pmatrix}}_{u}. \tag{3.11}$$

This is the **kinematic model** of the robot, with u a velocity-level quantity.

Driftless property. The kinematic model eq. (3.11) satisfies

$$u = 0 \implies \dot{q} = 0.$$

A system with this property is called **driftless**: it has no free evolution. When the inputs are zero, the state does not move.

Pseudovelocities. The vector u does not coincide with the actual control inputs of the robot (which are forces and torques), nor with the generalized velocities $\dot{q}$. It is a velocity-level quantity filtered through the kinematic constraints, and it is called **pseudo-velocity**. The kinematic model describes the motion as resulting from the pseudo-velocities u filtered through the constraints $A(q)$.

Non-uniqueness of the model. The choice of the basis $g_1, \dots, g_m$ is not unique, and different choices lead to equivalent models with different physical interpretations of u. In practice, we choose the basis so that the components of u have a natural meaning (e.g. linear and angular velocity for a unicycle).

3.7.1 Controllability and the Accessibility Rank Condition

Whether the constraints are holonomic or nonholonomic can be decided by studying the controllability of the kinematic model eq. (3.11). Two cases are possible:

- the system is **controllable**: for any two configurations $q_i, q_f \in C$ there exists an input $u(t)$ that steers the system from q_i to q_f while satisfying the constraints. The constraints do not affect the accessibility of C: they are (completely) nonholonomic;
- the system is **not controllable**: the constraints reduce the set of accessible configurations, so they are at least partially integrable.

To test controllability we need the Lie bracket of two vector fields,

$$[g_i, g_j](q) = \frac{\partial g_j}{\partial q}\, g_i(q) - \frac{\partial g_i}{\partial q}\, g_j(q), \tag{3.12}$$

which is a new vector field, in general not a linear combination of g_i and g_j. Let $\Delta = \operatorname{span}\{g_1, \dots, g_m\}$ be the distribution generated by the input vector fields. A distribution is **involutive** if it is closed under Lie bracket. The **accessibility distribution** $\Delta_{\mathcal{A}}$ is the involutive closure of Δ, i.e. the smallest involutive distribution containing it. It is built iteratively,

$$\Delta_1 = \Delta, \qquad \Delta_{i+1} = \Delta_i + \operatorname{span}\{[g, h] \ : \ g \in \Delta_1,\ h \in \Delta_i\},$$

until the dimension stops growing ($\Delta_{\mathcal{A}} = \Delta_i$ when $\Delta_{i+1} = \Delta_i$).

The controllability test is the **accessibility rank condition**:

$$\dim \Delta_{\mathcal{A}}(q) = n. \tag{3.13}$$

Let $\nu = \dim \Delta_{\mathcal{A}}$ and $m = \dim \Delta$. Three cases:

- $\nu = n$: eq. (3.13) holds, the system is controllable, the constraints are *completely nonholonomic*;
- $m < \nu < n$: the system is not controllable, but the loss of accessibility is not maximal; the constraints are *partially integrable* (the mechanical system is still nonholonomic);
- $\nu = m$: the system is not controllable and the loss of accessibility is maximal; the constraints are *completely integrable*, hence holonomic. This happens exactly when $\Delta_{\mathcal{A}} = \Delta$, i.e. when Δ is already involutive.

The **degree of nonholonomy** κ of a controllable system is the smallest integer such that $\dim \Delta_\kappa = n$, i.e. the minimum number of vector fields appearing in the nested brackets needed to generate $\mathbb{R}^n$. It quantifies *how* nonholonomic the constraints are.

Systems with a Single Input Vector Field

If the number of Pfaffian constraints is $k = n - 1$, then $m = 1$: the kinematic model has a single vector field, and Δ is always involutive (the bracket of a field with itself is zero). Hence a set of $n - 1$ Pfaffian constraints is always completely integrable. For example, a scalar Pfaffian constraint is always holonomic for a mechanical system with two generalized coordinates.

3.7.2 The Unicycle Model

Consider the rolling disk from section 3.6.3. Its constraint is eq. (3.9):

$$\underbrace{\begin{pmatrix}\sin\theta & -\cos\theta & 0\end{pmatrix}}_{A(q)}\begin{pmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\end{pmatrix} = 0.$$

The dimension of $\mathcal{N}(A)$ is $m = n - k = 3 - 1 = 2$. A basis is

$$g_1(q) = \begin{pmatrix}\cos\theta\\ \sin\theta\\ 0\end{pmatrix}, \qquad g_2(q) = \begin{pmatrix}0\\ 0\\ 1\end{pmatrix}.$$

The kinematic model is

$$\dot{q} = G(q)u = \begin{pmatrix}\cos\theta & 0\\ \sin\theta & 0\\ 0 & 1\end{pmatrix}\begin{pmatrix}u_1\\ u_2\end{pmatrix} \implies \begin{cases}\dot{x} = u_1\cos\theta,\\ \dot{y} = u_1\sin\theta,\\ \dot{\theta} = u_2.\end{cases} \tag{3.14}$$

The pseudo-velocity u_1 is the scalar velocity along the sagittal axis:

$$u_1 = \pm\sqrt{\dot{x}^2 + \dot{y}^2},$$

while $u_2 = \dot{\theta}$. Rename them:

$$v = u_1, \qquad \omega = u_2.$$

The linear velocity v is related to the rolling velocity ω_{roll} and the wheel radius by $v = \omega_{\text{roll}} \cdot r$. The model becomes

$$\dot{q} = \begin{pmatrix}\cos\theta\\ \sin\theta\\ 0\end{pmatrix} v + \begin{pmatrix}0\\ 0\\ 1\end{pmatrix}\omega. \tag{3.15}$$

Controllability. Compute the Lie bracket of the two input vector fields with eq. (3.12):

$$[g_1, g_2](q) = \frac{\partial g_2}{\partial q}g_1 - \frac{\partial g_1}{\partial q}g_2 = -\begin{pmatrix}0 & 0 & -\sin\theta\\ 0 & 0 & \cos\theta\\ 0 & 0 & 0\end{pmatrix}\begin{pmatrix}0\\ 0\\ 1\end{pmatrix} = \begin{pmatrix}\sin\theta\\ -\cos\theta\\ 0\end{pmatrix},$$

since g_2 is constant. This is the unit vector orthogonal to the sagittal axis, called the **slide vector field**: a velocity that is immediately unrealizable, but that specifies a direction achievable through a sequence of motions (a *wriggle*, see section 4.4). It is not a linear combination of g_1 and g_2. Hence the construction of the accessibility distribution ends immediately with $\Delta_{\mathcal{A}} = \Delta_2$ and

$$\dim\Delta_{\mathcal{A}} = \operatorname{rank}\begin{pmatrix}g_1 & g_2 & [g_1, g_2]\end{pmatrix} = 3 = n.$$

The unicycle is controllable with degree of nonholonomy $\kappa = 2$, and constraint eq. (3.9) is completely nonholonomic, as already shown constructively with the universal maneuver.

Robots Kinematically Equivalent to the Unicycle

A unicycle in the strict sense is not statically balanced. Two statically balanced structures are *kinematically* equivalent to it (their mechanics is different):

- **Differential drive**: with wheel radii r, angular velocities ω_R, ω_L, and distance d between the contact points,

$$v = r\frac{\omega_R + \omega_L}{2}, \qquad \omega = (\omega_R - \omega_L)\frac{r}{d}.$$

  There is a one-to-one correspondence between (v, ω) and (ω_R, ω_L). Here (x, y) is the midpoint between the contact points of the two fixed wheels and θ is the orientation of the body.

- **Synchro-drive**: the two motors directly control v and ω, common to the three steerable wheels, where (x, y) can be any point of the robot (e.g. the geometric center of the three contact points). Unlike the differential drive, the orientation of the body never changes, unless a third actuator is added.

3.7.3 The Bicycle Model

The bicycle has one fixed rear wheel and one steerable front wheel. Its configuration is $q = (x, y, \theta, \phi)^\top$, with (x, y) the position of the rear wheel contact point, θ the orientation of the body, and ϕ the steering angle. The configuration space is $C = \mathbb{R}^2 \times SO(2) \times SO(2)$, so $n = 4$.

Let l be the distance between the contact points of the two wheels. The position of the front wheel is

$$x_f = x + l\cos\theta, \tag{3.16}$$
$$y_f = y + l\sin\theta. \tag{3.17}$$

The two pure rolling constraints (rear and front wheel) are

$$\sin\theta\,\dot{x} - \cos\theta\,\dot{y} = 0, \tag{3.18}$$
$$\sin(\theta+\phi)\,\dot{x}_f - \cos(\theta+\phi)\,\dot{y}_f = 0. \tag{3.19}$$

Each wheel has its own zero-motion line. The two zero-motion lines meet at the **instantaneous center of rotation** (ICR), whose position changes with the configuration; every point of the vehicle moves instantaneously along an arc of circle centered at the ICR.

Substituting $\dot{x}_f, \dot{y}_f$, the second constraint becomes

$$\sin(\theta+\phi)\,\dot{x} - \cos(\theta+\phi)\,\dot{y} - l\cos\phi\,\dot{\theta} = 0.$$

In matrix form:

$$\underbrace{\begin{pmatrix} \sin\theta & -\cos\theta & 0 & 0 \\ \sin(\theta+\phi) & -\cos(\theta+\phi) & -l\cos\phi & 0 \end{pmatrix}}_{A(q)} \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\phi} \end{pmatrix} = 0.$$

The matrix has constant rank $k = 2$, so the null space has dimension $m = 4 - 2 = 2$. There are two choices of basis with a physical meaning, depending on which wheel is driven.

Rear-wheel drive. Look for a first vector of the form $g_1 = (\cos\theta, \sin\theta, \alpha, 0)^\top$ (motion of the rear wheel along its sagittal axis). Imposing the second constraint gives $\sin\phi - l\alpha\cos\phi = 0$, i.e. $\alpha = \frac{1}{l}\tan\phi$. With g_2 the pure steering velocity, a basis is

$$g_1^R(q) = \begin{pmatrix} \cos\theta \\ \sin\theta \\ \frac{1}{l}\tan\phi \\ 0 \end{pmatrix}, \qquad g_2^R(q) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix},$$

and the kinematic model is

$$\dot{q} = \begin{pmatrix} \cos\theta \\ \sin\theta \\ \frac{1}{l}\tan\phi \\ 0 \end{pmatrix} v + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \omega. \tag{3.20}$$

Here $v = \pm\sqrt{\dot{x}^2 + \dot{y}^2}$ is the velocity of the rear wheel contact point, and $\omega = \dot{\phi}$ is the steering velocity.

Singularity. The model eq. (3.20) is singular at $\phi = \pm\pi/2$, where $\tan\phi$ is undefined and g_1^R is not defined: the model assumes pure rolling, and at $\phi = \pm\pi/2$ any non-zero v would make the front wheel slide. In practice, the singularity is avoided by limiting the steering angle.

Controllability (rear-wheel drive). The first two brackets are

$$g_3^R = [g_1^R, g_2^R] = \begin{pmatrix} 0 \\ 0 \\ -\frac{1}{l\cos^2\phi} \\ 0 \end{pmatrix}, \qquad g_4^R = [g_1^R, g_3^R] = \begin{pmatrix} -\frac{\sin\theta}{l\cos^2\phi} \\ \frac{\cos\theta}{l\cos^2\phi} \\ 0 \\ 0 \end{pmatrix}.$$

They are obtained as follows. Since $g_2^R = e_4$ is constant, $g_3^R = -\partial g_1^R/\partial\phi$. The field g_3^R depends only on ϕ, and the ϕ-component of g_1^R is zero, so $(\partial g_3^R/\partial q)\, g_1^R = 0$; moreover g_3^R has only the θ-component, therefore $g_4^R = -(\partial g_1^R/\partial\theta)\, g_3^R$. The field g_3^R changes only the heading (a rotation in place of the vehicle), while g_4^R is parallel to the slide vector field of the unicycle (a lateral displacement of the rear wheel).

Since $g_1^R, g_2^R, g_3^R, g_4^R$ are linearly independent, with

$$\det\begin{pmatrix} g_1^R & g_2^R & g_3^R & g_4^R \end{pmatrix} = \frac{1}{l^2\cos^4\phi} \neq 0 \qquad (\phi \neq \pm\pi/2),$$

we have $\Delta_{\mathcal{A}}^R = \Delta_3^R$ and $\dim\Delta_{\mathcal{A}}^R = 4 = n$. The rear-wheel-drive bicycle is controllable with degree of nonholonomy $\kappa = 3$, and the rolling constraints are completely nonholonomic. The rank is lost exactly at the singularity of the model.

Front-wheel drive. If the robot is driven by the front wheel, replace g_1^R with $g_1^F = g_1^R\cos\phi$:

$$\dot{q} = \underbrace{\begin{pmatrix} \cos\theta\cos\phi \\ \sin\theta\cos\phi \\ \frac{1}{l}\sin\phi \\ 0 \end{pmatrix}}_{g_1^F} v + \underbrace{\begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}}_{g_2^F} \omega. \tag{3.21}$$

Now $v = \pm\sqrt{\dot{x}_f^2 + \dot{y}_f^2}$ is the velocity of the front wheel contact point, and the model is *not* singular at $\phi = \pm\pi/2$: in that situation a front-wheel-drive bicycle simply pivots around the rear

wheel. The brackets are

$$g_3^F = [g_1^F, g_2^F] = \begin{pmatrix} \cos\theta \sin\phi \\ \sin\theta \sin\phi \\ -\frac{\cos\phi}{l} \\ 0 \end{pmatrix}, \qquad g_4^F = [g_1^F, g_3^F] = \begin{pmatrix} -\frac{\sin\theta}{l} \\ \frac{\cos\theta}{l} \\ 0 \\ 0 \end{pmatrix}.$$

Also $g_1^F, g_2^F, g_3^F, g_4^F$ are linearly independent, so the front-wheel-drive bicycle is controllable with $\kappa = 3$.

Instantaneous Center of Rotation and Ackermann Steering

The two zero-motion lines (the directions in which each wheel cannot move) intersect at the **instantaneous center of rotation (ICR)**, the point around which the robot is rotating at a given instant. For a tricycle, the ICR is unique. For a car-like robot with two steerable front wheels, the two front zero-motion lines may intersect the rear zero-motion line at different points, so a single ICR does not exist. The **Ackermann steering** mechanism solves this by automatically orienting the front wheels so that their zero-motion lines intersect at a single ICR.

Kinematically Equivalent to the Bicycle

The bicycle is not statically balanced (its support polygon is a line segment). The **tricycle** is modeled as a bicycle by placing (x, y) at the midpoint between the two rear wheels, with θ the orientation of the vehicle and ϕ the steering angle. The **car-like** robot is modeled as a bicycle once Ackermann steering is in place. In both cases the kinematic model is eq. (3.20) or eq. (3.21), depending on whether the drive is on the rear or on the front wheels.

3.8 Chained Form

Many kinematic models of mobile robots can be recast in the same canonical form. This is useful because it allows the use of systematic procedures for planning and control. We limit the analysis to systems with two inputs, such as the unicycle and the bicycle.

Definition 3.5 (Chained Form)**.** *A $(2, n)$ **chained form** is the driftless system*

$$\begin{aligned} \dot{z}_1 &= w_1, \\ \dot{z}_2 &= w_2, \\ \dot{z}_3 &= z_2 w_1, \\ &\vdots \\ \dot{z}_n &= z_{n-1} w_1, \end{aligned} \tag{3.22}$$

with $z = (z_1, \ldots, z_n)$ the n-dimensional state and $w = (w_1, w_2)$ the 2-dimensional input.

Proposition 3.6. *The $(2, n)$ chained form is controllable, with degree of nonholonomy $\kappa = n - 1$.*

Proof. The input fields are $g_1 = (1, 0, z_2, \ldots, z_{n-1})^\top$ and $g_2 = e_2$. In the Jacobian of g_1, the entry on row $i \geq 3$ depends on z_{i-1}, so the column of z_j $(2 \leq j \leq n-1)$ is e_{j+1}. Since g_2 is constant,

$$[g_1, e_j] = -\frac{\partial g_1}{\partial z} e_j = -e_{j+1}, \qquad 2 \leq j \leq n-1.$$

Starting from $g_2 = e_2$, the iterated brackets are

$$\mathrm{ad}_{g_1}^k g_2 = (-1)^k e_{k+2}, \qquad k = 0, \ldots, n-2.$$

Together with g_1, which has first component 1, they span $\mathbb{R}^n$. Each bracket adds exactly one dimension, so the rank n is reached with brackets of length $n-1$ and not before: $\kappa = n - 1$. □

If w is constant, the chained form is a *linear* system (for w_1 constant, each equation $\dot{z}_i = z_{i-1}w_1$ is linear in z), so in digital control, where inputs are piecewise constant, the system is piecewise linear.

A generic n-dimensional two-input driftless system

$$\dot{q} = G(q)u = g_1(q)u_1 + g_2(q)u_2 \tag{3.23}$$

can be transformed into the chained form eq. (3.22) via a coordinate transformation and an input transformation,

$$z = \psi(q), \qquad u = \Gamma(q)\, w,$$

under necessary and sufficient conditions (not treated here). In particular, systems of the form eq. (3.23) with $n \leq 4$ can always be put in chained form. This applies to the unicycle and the bicycle, but the technique does not extend automatically to larger systems.

3.8.1 The Unicycle in Chained Form

Consider the unicycle model eq. (3.15). Use the coordinate transformation

$$z_1 = \theta, \qquad z_2 = x\cos\theta + y\sin\theta, \qquad z_3 = x\sin\theta - y\cos\theta, \tag{3.24}$$

and the input transformation

$$v = z_3 w_1 + w_2, \qquad \omega = w_1.$$

The model becomes the $(2,3)$ chained form

$$\dot{z}_1 = w_1, \tag{3.25}$$
$$\dot{z}_2 = w_2, \tag{3.26}$$
$$\dot{z}_3 = z_2 w_1. \tag{3.27}$$

Verification. First, $\dot{z}_1 = \dot{\theta} = \omega = w_1$. Then

$$\dot{z}_2 = \dot{x}\cos\theta + \dot{y}\sin\theta - x\sin\theta\,\dot{\theta} + y\cos\theta\,\dot{\theta} \tag{3.28}$$
$$= v - \underbrace{\dot{\theta}}_{\omega}\underbrace{(x\sin\theta - y\cos\theta)}_{z_3} = v - \omega z_3 = w_2, \tag{3.29}$$

and

$$\dot{z}_3 = \dot{x}\sin\theta - \dot{y}\cos\theta + x\cos\theta\,\dot{\theta} + y\sin\theta\,\dot{\theta} \tag{3.30}$$
$$= \underbrace{(\dot{x}\sin\theta - \dot{y}\cos\theta)}_{0 \text{ by the constraint}} + \dot{\theta}(x\cos\theta + y\sin\theta) = \omega\, z_2 = z_2 w_1. \tag{3.31}$$

Note that z_1 is simply the orientation θ, while z_2 and z_3 are the position of the unicycle in a frame whose first axis is the sagittal axis of the vehicle, $(\cos\theta, \sin\theta)$, and whose second axis is the right-hand normal $(\sin\theta, -\cos\theta)$ (fig. 3.5).

3.8.2 The Bicycle in Chained Form

The rear-wheel-drive bicycle model eq. (3.20) can also be transformed into chained form. Use the coordinate transformation

$$z_1 = x, \quad z_2 = \frac{1}{l}\sec^3\theta\tan\phi, \quad z_3 = \tan\theta, \quad z_4 = y,$$

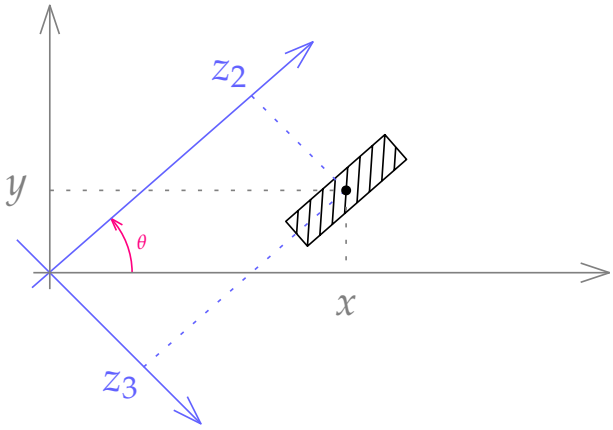


Figure 3.5: Interpretation of z_2 and z_3: coordinates of the unicycle in the frame aligned with the sagittal axis (first axis) and its right-hand normal (second axis).

and the input transformation

$$v = \frac{w_1}{\cos\theta}, \qquad \omega = -\frac{3}{l}\, w_1 \sec\theta \tan\theta \sin^2\phi + l\, w_2 \cos^3\theta \cos^2\phi.$$

The resulting system is the $(2, 4)$ chained form

$$\dot{z}_1 = w_1, \tag{3.32}$$
$$\dot{z}_2 = w_2, \tag{3.33}$$
$$\dot{z}_3 = z_2 w_1, \tag{3.34}$$
$$\dot{z}_4 = z_3 w_1. \tag{3.35}$$

The input transformation is undefined at $\cos\theta = 0$, i.e. at $\theta = \pi/2 + k\pi$: the equivalence between the two models is subject to $\theta \neq \pi/2 + k\pi$.

Verification. With $\dot{\theta} = v \tan\phi / l = w_1 \tan\phi / (l \cos\theta)$ and $\dot{\phi} = \omega$, the equations that do not involve ω are checked directly:

$$\dot{z}_1 = v \cos\theta = w_1, \tag{3.36}$$
$$\dot{z}_3 = \sec^2\theta\, \dot{\theta} = \frac{1}{l} \sec^3\theta \tan\phi\, w_1 = z_2 w_1, \tag{3.37}$$
$$\dot{z}_4 = v \sin\theta = w_1 \tan\theta = z_3 w_1. \tag{3.38}$$

The input ω enters only through $\dot{z}_2$:

$$\dot{z}_2 = \frac{3}{l^2} \sec^4\theta \tan\theta \tan^2\phi\, w_1 + \frac{1}{l} \sec^3\theta \sec^2\phi\, \omega.$$

Imposing $\dot{z}_2 = w_2$ and solving for ω, with $\cos^3\theta \sec^4\theta = \sec\theta$ and $\cos^2\phi \tan^2\phi = \sin^2\phi$, gives the input transformation above.

Trailed Vehicles

An interesting property of **trailed vehicles** emerges when considering a car-like robot: if the trailer is hitched exactly at the midpoint between the two rear wheels, the kinematic model can still be converted into chained form. This property holds recursively: if the trailer itself has two rear wheels and an additional trailer is hitched at their midpoint, the overall kinematic model can still be expressed in chained form.

3.9 Takeaways

1. A wheeled robot is in **static balance** if the projection of its center of mass falls inside the **support polygon**, the convex hull of its contact points. Unicycle and bicycle are not balanced; differential drive, synchro-drive, tricycle and car-like robots are.

2. Three types of wheels exist: **fixed**, **steerable**, and **caster**. They differ in whether they can steer, whether they are actuated, and whether they are kinematically "invisible".

3. The main kinematic structures are the **differential drive**, the **synchro-drive**, the **tricycle**, and the **car-like** robot.

4. The **mechanical differential** allows two wheels on the same axle to rotate at different speeds, avoiding slip during turns.

5. Geometric constraints $h(q) = 0$ reduce the configuration space to a sub-manifold; kinematic constraints in Pfaffian form $A(q)\dot{q} = 0$ restrict the admissible velocities to $\mathcal{N}(A(q))$. Integrable kinematic constraints are holonomic, the others are nonholonomic.

6. The kinematic model $\dot{q} = G(q)u$ is driftless, and u are pseudo-velocities. The constraints are completely nonholonomic if and only if the accessibility rank condition $\dim \Delta_{\mathcal{A}} = n$ holds, i.e. if the model is controllable.

7. The unicycle ($\kappa = 2$) and the bicycle ($\kappa = 3$, singular at $\phi = \pm\pi/2$ only in the rear-wheel-drive case) are controllable; differential drive and synchro-drive are kinematically equivalent to the unicycle, tricycle and car-like to the bicycle.

8. Both can be put in **chained form**, always controllable with $\kappa = n - 1$, which allows systematic planning and control.

Chapter 4

Nonlinear Controllability in Depth

4.1 Scope of This Chapter

chapter 3 introduced the controllability test as an operational recipe: build the accessibility distribution with the Lie bracket eq. (3.12), check the rank condition eq. (3.13), and read the degree of nonholonomy κ (section 3.7.1). The kinematic models, their controllability, and the chained-form transformations, with the related computations, are in that chapter.

This chapter does *not* repeat those definitions and results. It answers the question that the recipe leaves open: *why does it work?* It collects

- the formal statement of the controllability criterion and an outline of its proof (section 4.2);
- the properties of the Lie bracket and the derivation of its geometric meaning (sections 4.3 and 4.4);
- the link between involutivity, integrability and holonomy (section 4.5);
- exercises with solutions, including the bracket computations for the vehicles of chapter 3 (section 4.6).

Throughout, the state is denoted by $x \in \mathbb{R}^n$ when talking about a generic system, and by q for the vehicles. The system is the driftless one of eq. (3.11),

$$\dot{x} = \sum_{i=1}^{m} g_i(x)\, u_i. \tag{4.1}$$

4.2 Controllability of Driftless Systems

Definition 4.1 (Controllability of a Nonlinear System)**.** *The system eq. (4.1) is* ***controllable*** *if, for any pair of states (x_i, x_f), with $x(0) = x_i$, there exists a profile $\tilde{u}(t)$ for the inputs on $t \in [0, T]$ such that the system reaches $x(T) = x_f$.*

For linear systems the test is a rank condition on the controllability matrix. For nonlinear systems the test is again a rank condition, but on a matrix built from Lie brackets, and the rank condition is, in general, only a statement about *accessibility*, i.e. about the reachable set having non-empty interior. Driftless systems are special: the two notions coincide. This is the content of the following theorem (stated here, and used in chapter 3, as the accessibility rank condition).

Theorem 4.2 (Accessibility Criterion (Chow–Rashevskii))**.** *Consider the driftless system eq. (4.1) on a connected domain, and let $\Delta_{\mathcal{A}}$ be its accessibility distribution (section 3.7.1).*

1. *If $\dim \Delta_{\mathcal{A}}(x) = n$ at every x, the system is controllable.*

2. *Conversely, if the system is analytic and* $\dim \Delta_{\mathcal{A}} = \nu < n$*, it is not controllable: from any initial state the reachable set is contained in a* ν*-dimensional sub-manifold.*

Drift

If the system has a drift term, $\dot{x} = f(x) + \sum_i g_i(x)u_i$, the rank condition on $\Delta_{\mathcal{A}}$ only guarantees accessibility: one cannot reverse the motion along f, so the reachable set need not be a neighbourhood. The wheeled-robot kinematic models in this course are driftless, so the criterion above is a necessary and sufficient test.

Outline of the proof. The proof of part 1 has three steps.

1. *Brackets are reachable directions.* By section 4.4, a cyclic input sequence of duration 4ε produces a displacement $\varepsilon^2[g_i, g_j] + O(\varepsilon^3)$. Iterating the construction gives the same for nested brackets, with displacement of order ε^k for a bracket of length k.

2. *Full rank gives an open set.* If the vector fields and their brackets span $\mathbb{R}^n$ at x_0, suitable combinations of such sequences move the state in n independent directions, so the reachable set from x_0 contains an open neighbourhood of x_0.

3. *Globalization.* The system is driftless, so every motion can be reversed with the opposite inputs: "being reachable from" is an equivalence relation. Its classes are open by step 2, hence they also are closed (the complement is a union of open classes). On a connected domain there is only one class: the whole domain.

Part 2 follows from the Frobenius theorem applied to the (involutive) distribution $\Delta_{\mathcal{A}}$, see section 4.5.

Holonomic vs. nonholonomic, revisited. Combining theorem 4.2 with the classification of section 3.7.1: the kinematic model of a robot is controllable if and only if its kinematic constraints are *completely* nonholonomic ($\nu = n$). Partially integrable constraints are still nonholonomic, but the system is not controllable.

4.3 Lie Brackets in Depth

The definition of the Lie bracket is eq. (3.12). Using the Jacobians $J_i(x) = \partial g_i/\partial x$, it reads $[g_1, g_2] = J_2 g_1 - J_1 g_2$.

Algebraic properties. For vector fields f, g, h and constants a, b:

$$[f, g] = -[g, f] \qquad \text{(antisymmetry; in particular } [f, f] = 0\text{)}, \tag{4.2}$$

$$[af + bg, h] = a[f, h] + b[g, h] \qquad \text{(bilinearity)}, \tag{4.3}$$

$$[f, [g, h]] + [g, [h, f]] + [h, [f, g]] = 0 \qquad \text{(Jacobi identity)}. \tag{4.4}$$

Antisymmetry is immediate from the definition and is the reason why, in the accessibility distribution, each pair g_i, g_j is counted only once. Bilinearity follows from the linearity of the Jacobian. The Jacobi identity is verified by expanding the six terms: the second-derivative terms cancel in pairs by the symmetry of the Hessian. Its practical consequence is that, to span all brackets of a given length, it is enough to consider *left-nested* brackets $[g_{i_1}, [g_{i_2}, [\ldots, g_{i_k}]]]$.

Example in $\mathbb{R}^3$. Let

$$g_1 = \begin{pmatrix} \cos x_3 \\ \sin x_3 \\ 0 \end{pmatrix}, \qquad g_2 = \begin{pmatrix} 0 \\ 0 \\ x_1 \end{pmatrix}.$$

The Jacobians are

$$J_1 = \begin{pmatrix} 0 & 0 & -\sin x_3 \\ 0 & 0 & \cos x_3 \\ 0 & 0 & 0 \end{pmatrix}, \qquad J_2 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix},$$

hence

$$[g_1, g_2](x) = J_2 g_1 - J_1 g_2 = \begin{pmatrix} 0 \\ 0 \\ \cos x_3 \end{pmatrix} - \begin{pmatrix} -x_1 \sin x_3 \\ x_1 \cos x_3 \\ 0 \end{pmatrix} = \begin{pmatrix} x_1 \sin x_3 \\ -x_1 \cos x_3 \\ \cos x_3 \end{pmatrix}.$$

The determinant of $(g_1, g_2, [g_1, g_2])$ is x_1^2, so $[g_1, g_2]$ is linearly independent from g_1, g_2 only for $x_1 \neq 0$: on the plane $x_1 = 0$ the rank condition fails at the first bracket, and one would have to compute higher-order brackets.

4.4 Interpretation of the Lie Bracket

Consider the 2-input system $\dot{x} = g_1(x)u_1 + g_2(x)u_2$ and apply the following input profile on $t \in [0, 4\varepsilon]$:

$$(u_1, u_2) = (1, 0) \qquad t \in [0, \varepsilon], \tag{4.5}$$
$$(u_1, u_2) = (0, 1) \qquad t \in [\varepsilon, 2\varepsilon], \tag{4.6}$$
$$(u_1, u_2) = (-1, 0) \qquad t \in [2\varepsilon, 3\varepsilon], \tag{4.7}$$
$$(u_1, u_2) = (0, -1) \qquad t \in [3\varepsilon, 4\varepsilon]. \tag{4.8}$$

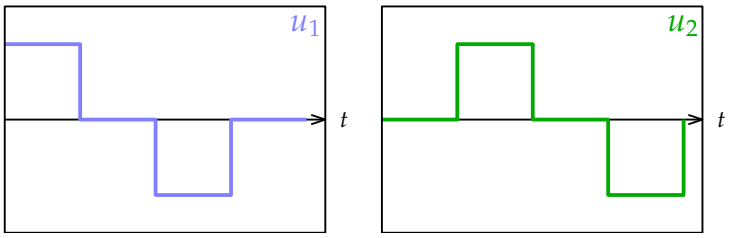


Figure 4.1: Input profile for the interpretation of the Lie bracket.

Proposition 4.3 (Wriggle)**.** *After this input sequence,*

$$x(4\varepsilon) = x(0) + \varepsilon^2 [g_1, g_2](x(0)) + O(\varepsilon^3). \tag{4.9}$$

Proof. The flow of a vector field g for a time t is, to second order,

$$\phi_t^g(x) = x + t\, g(x) + \tfrac{t^2}{2} J_g(x)\, g(x) + O(t^3),$$

and the flow for $-g$ is obtained with $t \to -t$. Write g_i, J_i for the fields and Jacobians evaluated at $x(0)$, and follow the four phases.

$$x_1 = x + \varepsilon g_1 + \tfrac{\varepsilon^2}{2} J_1 g_1,$$
$$x_2 = x_1 + \varepsilon g_2(x_1) + \tfrac{\varepsilon^2}{2} J_2 g_2 = x + \varepsilon(g_1 + g_2) + \varepsilon^2 \left(\tfrac{1}{2} J_1 g_1 + J_2 g_1 + \tfrac{1}{2} J_2 g_2\right),$$

using $g_2(x_1) = g_2 + \varepsilon J_2 g_1 + O(\varepsilon^2)$. In the third phase, $g_1(x_2) = g_1 + \varepsilon J_1(g_1 + g_2) + O(\varepsilon^2)$, so

$$\begin{aligned} x_3 &= x_2 - \varepsilon g_1(x_2) + \tfrac{\varepsilon^2}{2} J_1 g_1 \\ &= x + \varepsilon g_2 + \varepsilon^2 (J_2 g_1 - J_1 g_2 + \tfrac{1}{2} J_2 g_2), \end{aligned}$$

where the $J_1 g_1$ terms cancel ($\frac{1}{2} - 1 + \frac{1}{2} = 0$). Finally, with $g_2(x_3) = g_2 + \varepsilon J_2 g_2 + O(\varepsilon^2)$,

$$x_4 = x_3 - \varepsilon g_2(x_3) + \tfrac{\varepsilon^2}{2} J_2 g_2 = x + \varepsilon^2 (J_2 g_1 - J_1 g_2) + O(\varepsilon^3) = x + \varepsilon^2 [g_1, g_2] + O(\varepsilon^3). \qquad \square$$

The displacement is in the direction of the Lie bracket, with an error $O(\varepsilon^3)$ that becomes negligible as $\varepsilon \to 0$. A suitable cyclic sequence of inputs (a *wriggle*) therefore produces a net motion along a direction that is not directly spanned by the inputs. Note the scaling: the displacement is of order ε^2 for a maneuver of duration 4ε, so motion along a bracket is "slower" than motion along the input fields, and brackets of length k are slower still (order ε^k).

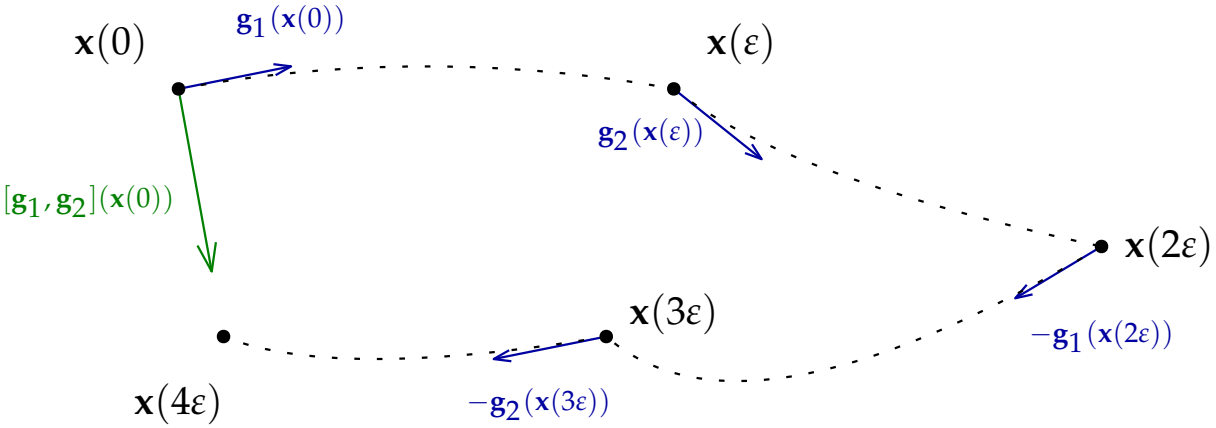


Figure 4.2: The Lie bracket gives the direction of the net displacement produced by the cyclic input sequence.

A Numerical Check of the Wriggle

For the unicycle at $\theta = 0$, apply the sequence $(v, \omega) = (1, 0), (0, 1), (-1, 0), (0, -1)$, each for a time ε. The vehicle moves forward by ε, turns left by ε, moves backward by ε along the new heading, and turns back. The final position is $(\varepsilon - \varepsilon\cos\varepsilon,\ -\varepsilon\sin\varepsilon) \approx (0, -\varepsilon^2)$ with $\theta = 0$, i.e. $\varepsilon^2[g_1, g_2]$ with $[g_1, g_2] = (\sin\theta, -\cos\theta, 0)^\top$ the slide vector field of chapter 3, in agreement with eq. (4.9): a lateral displacement obtained with no lateral velocity.

Lie Bracket as a Wriggle

The Lie bracket describes a motion that cannot be produced by any single input, but can be produced by a sufficiently fast cyclic sequence of inputs. For a car-like robot, this is exactly how you parallel-park: no single input moves the car sideways, but a cyclic sequence of forward/backward and steering commands does.

4.5 Distributions and Integrability

Definition 4.4 (Distribution)**.** *Let C be a smooth manifold. A **distribution** Δ is a map that assigns to each point $x \in C$ a subspace of the tangent space $T_x C$:*

$$\Delta : x \mapsto \Delta(x) \subset T_x C.$$

*It has **dimension** k, written $\dim \Delta = k$, if $\dim \Delta(x) = k$ for every $x \in C$.*

For the system eq. (4.1) the input distribution is $\Delta = \operatorname{span}\{g_1, \dots, g_m\}$, and it is a distribution in the sense above whenever the g_i are independent at every point.

Definition 4.5 (Involutive Distribution)**.** *A distribution Δ is **involutive** if it is closed under Lie bracket: for any vector fields g, h with values in Δ, also $[g, h]$ takes values in Δ.*

The accessibility distribution $\Delta_{\mathcal{A}}$ is, by construction, the smallest involutive distribution containing Δ: its *involutive closure*.

Theorem 4.6 (Frobenius)**.** *A distribution of constant dimension k is **integrable**, i.e. through every point there passes a k-dimensional sub-manifold tangent to it, if and only if it is involutive.*

Frobenius' theorem is the bridge to section 3.6. Let $\Delta = \mathcal{N}(A(q))$ be the distribution of the admissible velocities of the Pfaffian constraints $A(q)\dot{q} = 0$. Then:

- Δ involutive $\iff \Delta_{\mathcal{A}} = \Delta \iff \nu = m$: the constraints are completely integrable, hence holonomic, and the motion is confined to the integral sub-manifolds of Δ;
- Δ not involutive: the constraints are nonholonomic. The sub-manifold tangent to $\Delta_{\mathcal{A}}$ (when $\nu < n$) is the set of accessible configurations.

The case $k = n - 1$ of section 3.7.1 is immediate: a distribution generated by a single vector field is always involutive, since $[g, g] = 0$.

Proposition 4.7. *Let $\Delta_1 = \Delta$ and $\Delta_{i+1} = \Delta_i + \operatorname{span}\{[g, h] : g \in \Delta_1,\ h \in \Delta_i\}$, at points where the dimensions are constant. Then the chain stabilizes after at most $n - m$ steps, and $\Delta_{\mathcal{A}} = \Delta_{n-m+1}$. In particular, brackets of length greater than $n - m + 1$ are never needed, and $\kappa \le n - m + 1$.*

Proof. The chain is increasing, $\Delta_i \subseteq \Delta_{i+1}$. If $\Delta_{i+1} = \Delta_i$, then also $\Delta_{i+2} = \Delta_{i+1}$, because the new brackets involve only elements of $\Delta_{i+1} = \Delta_i$, which are already in Δ_{i+1}. So the chain is either strictly increasing or constant from some point on. Its dimension starts at m and cannot exceed n: a strict increase of at least one dimension per step can happen at most $n - m$ times. (Analyticity is not needed for this argument; it is needed for the converse in theorem 4.2.) □

For $m = 2$ the bound is $n - 1$: the unicycle ($n = 3$, $\kappa = 2$), the bicycle ($n = 4$, $\kappa = 3$) and the chained form ($\kappa = n - 1$, see proposition 3.6) all attain it.

4.6 Exercises

1. **Heisenberg system.** Let $g_1 = (1, 0, -x_2/2)^\top$ and $g_2 = (0, 1, x_1/2)^\top$ on $\mathbb{R}^3$. Compute $[g_1, g_2]$, and decide whether $\dot{x} = g_1 u_1 + g_2 u_2$ is controllable. What is κ?
2. **Unicycle.** Compute $[g_1, g_2]$ for eq. (3.15) from the Jacobians, and the determinant of $(g_1, g_2, [g_1, g_2])$.
3. **Rear-wheel-drive bicycle.** For eq. (3.20), compute $g_3 = [g_1, g_2]$, $g_4 = [g_1, g_3]$, and the determinant of (g_1, g_2, g_3, g_4). For which ϕ does the rank drop?
4. **Front-wheel-drive bicycle.** Repeat the computation for eq. (3.21). Does the rank drop at $\phi = \pm\pi/2$?
5. **Chained form.** For the $(2, 4)$ chained form compute $[g_1, g_2]$ and $[g_1, [g_1, g_2]]$, and verify that the rank is 4 and $\kappa = 3$.
6. **Partial integrability.** Consider on $\mathbb{R}^4$ the fields $g_1 = (1, 0, -x_2/2, 0)^\top$, $g_2 = (0, 1, x_1/2, 0)^\top$. Compute ν and m, classify the constraints with the cases of section 3.7.1, and say whether the system is controllable.

7. **Algebra of the bracket.** Prove that $[f, g] = -[g, f]$ and $[f, f] = 0$, and that for $f(x) = Ax$, $g(x) = Bx$ (linear fields) one has $[f, g](x) = (BA - AB)x$.

8. **Numerical wriggle.** Simulate the unicycle with the sequence of the numerical check in section 4.4, for $\varepsilon = 0.1, 0.05, 0.025$. Verify that the final lateral displacement scales as ε^2.

Solutions (sketch).

1. $[g_1, g_2] = (0, 0, 1)^\top$. The matrix $(g_1, g_2, [g_1, g_2])$ has determinant 1 everywhere, so the system is controllable with $\kappa = 2$. (This is the standard model of a nonholonomic integrator.)

2. Only $\partial g_1/\partial q$ is non-zero, with the column of θ equal to $(-\sin\theta, \cos\theta, 0)^\top$. Hence $[g_1, g_2] = -(\partial g_1/\partial q) g_2 = (\sin\theta, -\cos\theta, 0)^\top$, and the determinant is 1 for every θ.

3. $g_3 = -\partial g_1/\partial\phi = (0, 0, -1/(l\cos^2\phi), 0)^\top$ and $g_4 = (-\sin\theta, \cos\theta, 0, 0)^\top/(l\cos^2\phi)$. The determinant is $1/(l^2\cos^4\phi)$: the rank drops only at $\phi = \pm\pi/2$, the singularity of the model.

4. $g_3^F = (\cos\theta\sin\phi,\ \sin\theta\sin\phi,\ -\cos\phi/l,\ 0)^\top$ and $g_4^F = (-\sin\theta,\ \cos\theta,\ 0,\ 0)^\top/l$. In the frame aligned with the heading the determinant has modulus $1/l^2$, independent of ϕ: the rank never drops, and $\kappa = 3$.

5. $g_1 = (1, 0, z_2, z_3)^\top$, $g_2 = e_2$, $[g_1, g_2] = -e_3$, $[g_1, [g_1, g_2]] = e_4$. The four fields $g_1, g_2, -e_3, e_4$ are independent.

6. $[g_1, g_2] = (0, 0, 1, 0)^\top = e_3$, and the next brackets vanish: $\nu = 3$, $m = 2$, $n = 4$, so $m < \nu < n$: the constraints are partially integrable. The system is not controllable: x_4 is constant (the constraint $\dot{x}_4 = 0$ is integrable).

7. Antisymmetry is immediate from $J_g f - J_f g$. For the linear fields, $J_f = A$, $J_g = B$, so $[f, g] = BAx - ABx$.

8. The lateral displacement is $-\varepsilon\sin\varepsilon \approx -\varepsilon^2$: halving ε divides it by about 4.

4.7 Takeaways

1. For driftless systems the accessibility rank condition is necessary and sufficient for controllability (Chow–Rashevskii); with drift it only guarantees accessibility.

2. The Lie bracket is bilinear, antisymmetric, and satisfies the Jacobi identity. A cyclic input sequence of duration 4ε produces a displacement $\varepsilon^2[g_1, g_2] + O(\varepsilon^3)$ (a **wriggle**).

3. Frobenius: a constant-dimension distribution is integrable if and only if it is involutive. Holonomic constraints correspond to $\Delta_{\mathcal{A}} = \Delta$.

4. The chain of brackets stabilizes after at most $n - m$ steps, so $\kappa \le n - m + 1$: the unicycle, the bicycle and the chained form (chapter 3) all attain the bound for $m = 2$.

Chapter 5

Path and Trajectory Planning

Once the kinematic model of a wheeled mobile robot is known, the planning problem becomes: find a path $q(s)$ in the configuration space that respects the kinematic constraints, and then a timing law $s(t)$ that makes it executable by the robot. This chapter introduces the geometric model, the notion of differential flatness, two algorithms for path planning (one based on polynomial interpolation of the flat outputs, one based on parametrized inputs on the chained form), and the timing law that turns a path into an executable trajectory.

5.1 Path vs Trajectory

We want to transfer a mobile robot from an initial configuration q_i to a final configuration q_f without violating the kinematic constraints on $\dot{q}$. In general, we separate the geometric path from the timing law:

$$q(s), \quad s \in [s_i, s_f], \tag{5.1}$$
$$s(t), \quad t \in [t_i, t_f]. \tag{5.2}$$

The path parameter s is usually chosen as the arc length.

- **Path planning**: plan a geometric path $q(s)$ in the configuration space.
- **Trajectory planning**: plan a time evolution $q(t)$. This is usually done indirectly: first plan a path, then add a timing law.

A trajectory $q(s(t))$ must satisfy the kinematic constraints:

$$\dot{q} = \frac{dq}{dt} = \frac{dq}{ds}\dot{s} = q'\dot{s}, \tag{5.3}$$

where q' denotes the derivative of q with respect to s. This is the *tangent vector* of the geometric path in the configuration space. If s is the arc length, then $\|q'\| = 1$.

Example. Consider a robot with $q = (x, y)$, and suppose we want to go from $(1, 0)$ to $(0, 1)$. One possible choice for the geometric path is

$$q(s) = \begin{pmatrix} \cos s \\ \sin s \end{pmatrix}, \qquad s \in \left[0, \frac{\pi}{2}\right].$$

This is exactly the arc length, because

$$q'(s) = \begin{pmatrix} -\sin s \\ \cos s \end{pmatrix} \implies \|q'(s)\| = \sqrt{\sin^2 s + \cos^2 s} = 1.$$

Another parametrization is

$$q(s) = \begin{pmatrix} \sqrt{1-s^2} \\ s \end{pmatrix}, \qquad s \in [0,1],$$

but here s is not the arc length:

$$q'(s) = \begin{pmatrix} -\frac{s}{\sqrt{1-s^2}} \\ 1 \end{pmatrix} \implies \|q'(s)\| = \frac{1}{\sqrt{1-s^2}}.$$

5.2 The Geometric Model

The kinematic constraints are $A(q)\dot{q} = 0$. Substituting $\dot{q} = q'\dot{s}$, we get

$$A(q)q'\dot{s} = 0.$$

Assuming $\dot{s} \neq 0$, the tangent vector must satisfy

$$q' \in \mathcal{N}(A(q)).$$

Hence q' can be expressed as a linear combination of the basis vectors $g_1, \ldots, g_m$, exactly as in the kinematic model. We obtain the **geometric model**:

$$q' = G(q)\tilde{u}, \tag{5.4}$$

which describes explicitly the admissible tangent vectors. $\tilde{u}$ is the vector of **geometric inputs**. Multiplying both sides by $\dot{s}$ gives

$$\underbrace{u(t)}_{\text{pseudo-velocity}} = \tilde{u}(s)\dot{s}(t).$$

After choosing a profile for the geometric inputs $\tilde{u}(s)$, we have determined a geometric path in the configuration space. Only after defining also $s(t)$ we obtain a particular trajectory.

Example: unicycle. For the unicycle model, the pure rolling constraint on the tangent vector is

$$\begin{pmatrix} \sin\theta & -\cos\theta & 0 \end{pmatrix} q' = 0.$$

This means that the tangent vector q' must always be aligned with the sagittal axis of the unicycle. Paths such as the one in fig. 5.1 are not admissible.

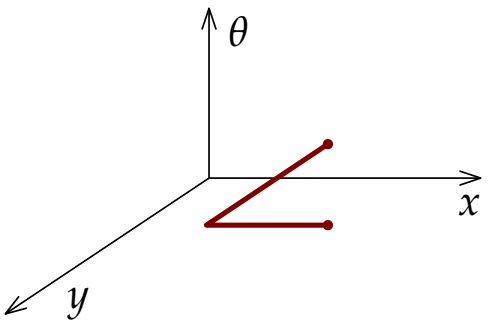


Figure 5.1: Geometric path with tangent vector not aligned with the sagittal axis.

5.3 Differential Flatness

Definition 5.1 (Differential Flatness)**.** *Consider a general nonlinear system*

$$\dot{x} = f(x) + G(x)u.$$

The system is ***differentially flat*** *if there exists a vector* $y = \Psi(x)$*, called the* ***flat outputs****, such that the state and the control inputs can be expressed algebraically as a function of* y *and its time derivatives up to a finite order:*

$$x = \Phi_x(y, \dot{y}, \ddot{y}, \dots, y^{(r_x)}), \tag{5.5}$$

$$u = \Phi_u(y, \dot{y}, \ddot{y}, \dots, y^{(r_u)}). \tag{5.6}$$

The same definition holds if all derivatives are taken with respect to the path parameter s instead of time, as in the geometric model. Flatness is useful in control: if we define a trajectory for y that is differentiable up to the order $\max(r_x, r_u)$, we can reconstruct the state evolution x and the required inputs u through the reconstruction maps Φ_x, Φ_u.

The unicycle is flat. For the unicycle model,

$$x' = \tilde{v}\cos\theta, \tag{5.7}$$

$$y' = \tilde{v}\sin\theta, \tag{5.8}$$

$$\theta' = \tilde{\omega}. \tag{5.9}$$

The flat outputs are the Cartesian coordinates (x, y). If we know the path of (x, y), we can infer the path of the orientation θ and the input profile that executes it. At any $s = s^*$, the orientation must be aligned with the direction of motion, i.e. the vector $(x'(s^*), y'(s^*))$:

$$\theta(s) = \operatorname{atan2}(y'(s), x'(s)) + k\pi, \tag{5.10}$$

where $k \in \{0, 1\}$ defines the direction of motion (forward or backward).

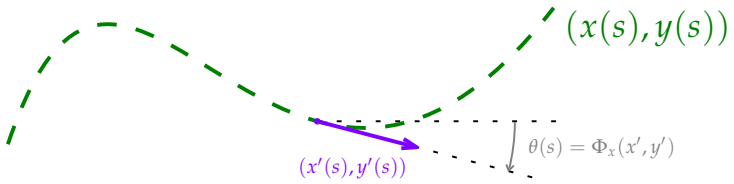


Figure 5.2: Interpretation of the flat outputs (x, y) for the unicycle: the orientation is determined by the tangent to the Cartesian path.

The input $\tilde{v}$ is the norm of the Cartesian velocity:

$$\tilde{v} = \pm\sqrt{(x')^2 + (y')^2}, \tag{5.11}$$

and the angular velocity is

$$\tilde{\omega} = \frac{y''x' - x''y'}{(x')^2 + (y')^2}. \tag{5.12}$$

Equations (5.11) and (5.12) are the reconstruction map for the input $\tilde{u} = \Phi_u(x', y', x'', y'')$. They have a singularity at $x' = y' = 0$: atan2 is undefined and $\tilde{\omega}$ has a zero denominator. Intuitively, if the robot stops, we cannot know whether it is steering in place or not.

Flatness and Chained-Form

A driftless dynamic system is differentially flat if and only if it can be transformed into chained-form. Since all the wheeled mobile robots of interest admit a chained-form representation, they are also flat.

The bicycle is flat. For the bicycle model,

$$x' = \tilde{v}\cos\theta, \tag{5.13}$$
$$y' = \tilde{v}\sin\theta, \tag{5.14}$$
$$\theta' = \frac{\tilde{v}}{l}\tan\phi, \tag{5.15}$$
$$\phi' = \tilde{\omega}. \tag{5.16}$$

The coordinates x, y, θ are reconstructed as in the unicycle case. The steering angle ϕ is reconstructed from θ':

$$\theta(s) = \operatorname{atan2}(y', x') + k\pi, \tag{5.17}$$
$$\theta'(s) = \frac{x'y'' - y'x''}{(x')^2 + (y')^2}, \tag{5.18}$$

hence

$$\phi(s) = \arctan\left(\frac{l\theta'}{\tilde{v}}\right) \tag{5.19}$$
$$= \arctan\left(\pm\frac{l(x'y'' - y'x'')}{((x')^2 + (y')^2)^{3/2}}\right). \tag{5.20}$$

Flat outputs for chained-form systems. Recall the chained form from section 3.8. For a 2-input, 3-state chained-form system:

$$z_1' = \tilde{w}_1, \tag{5.21}$$
$$z_2' = \tilde{w}_2, \tag{5.22}$$
$$z_3' = z_2\tilde{w}_1. \tag{5.23}$$

The flat outputs are always the first and the last variables:

$$y = \begin{pmatrix} z_1 \\ z_3 \end{pmatrix}.$$

The reconstruction maps are

$$\Phi_x(z_1', z_3') = \frac{z_3'}{z_1'}, \tag{5.24}$$
$$\Phi_u(z_1', z_1'', z_3', z_3'') = \begin{cases} \tilde{w}_1 = z_1', \\ \tilde{w}_2 = \dfrac{z_1'z_3'' - z_1''z_3'}{(z_1')^2}. \end{cases} \tag{5.25}$$

5.4 Path Planning for the Unicycle

Any flat-output profile is automatically mapped to an admissible path. The algorithm for path planning exploiting differential flatness is the following:

1. Compute the initial and final values y_i, y_f of the flat outputs from q_i, q_f.
2. Define $y(s)$ by interpolating y_i, y_f with the appropriate boundary conditions for q.
3. Use the reconstruction formulas to find $q(s)$ and $\tilde{u}(s)$.

The same algorithm applies to trajectory planning by interpolating over t instead of s.

Apply this to the unicycle. Use $s \in [0, 1]$. From

$$q_i = \begin{pmatrix} x_i \\ y_i \\ \theta_i \end{pmatrix}, \qquad q_f = \begin{pmatrix} x_f \\ y_f \\ \theta_f \end{pmatrix},$$

extract

$$y_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix}, \qquad y_f = \begin{pmatrix} x_f \\ y_f \end{pmatrix},$$

and interpolate with a cubic polynomial:

$$x(s) = s^3 x_f - (s-1)^3 x_i + \alpha_x s^2 (s-1) + \beta_x s (s-1)^2, \tag{5.26}$$
$$y(s) = s^3 y_f - (s-1)^3 y_i + \alpha_y s^2 (s-1) + \beta_y s (s-1)^2. \tag{5.27}$$

This polynomial already satisfies the boundary conditions

$$x(0) = x_i, \quad x(1) = x_f, \quad y(0) = y_i, \quad y(1) = y_f.$$

The parameters $\alpha_x, \beta_x, \alpha_y, \beta_y$ are chosen to satisfy two additional boundary conditions from the kinematic constraints: the tangent (x', y') must be aligned with θ_i at $s = 0$ and with θ_f at $s = 1$. Hence

$$\operatorname{atan2}(y'(0), x'(0)) = \theta_i, \tag{5.28}$$
$$\operatorname{atan2}(y'(1), x'(1)) = \theta_f. \tag{5.29}$$

The initial and final Cartesian velocities must be

$$x'(0) = k_i \cos\theta_i, \qquad x'(1) = k_f \cos\theta_f, \tag{5.30}$$
$$y'(0) = k_i \sin\theta_i, \qquad y'(1) = k_f \sin\theta_f, \tag{5.31}$$

with k_i, k_f arbitrary non-zero constants; for simplicity set $k_i = k_f = k$. Solving algebraically for $\alpha_x, \beta_x, \alpha_y, \beta_y$:

$$\alpha_x = k\cos\theta_f - 3x_f, \qquad \beta_x = k\cos\theta_i + 3x_i, \tag{5.32}$$
$$\alpha_y = k\sin\theta_f - 3y_f, \qquad \beta_y = k\sin\theta_i + 3y_i. \tag{5.33}$$

Since $\tilde{v}(0) = \tilde{v}(1) = k$, the parameter k shapes the trajectory by affecting the length of the first and last "linear" segments of the path, as shown in fig. 5.3.

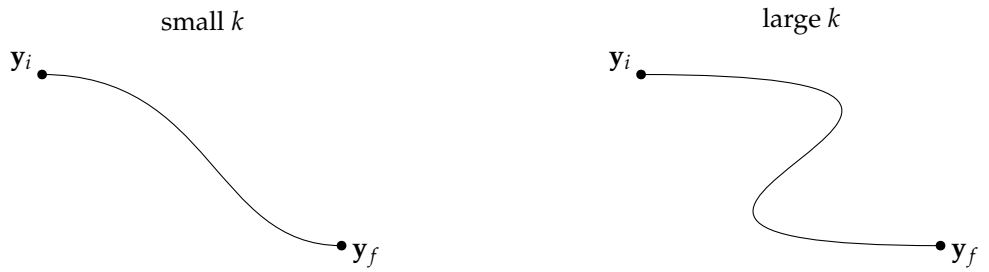


Figure 5.3: Difference between two paths with the same interpolating points but different choice of k.

5.5 Path Planning for a Chained-Form System

Apply the same algorithm to the chained form eq. (3.22). We illustrate it for the $(2, 3)$ case, which covers the unicycle (section 3.8.1); the $(2, 4)$ case of the bicycle (section 3.8.2) follows identically.

The flat outputs are z_1, z_3, hence interpolate (z_{1i}, z_{3i}) to (z_{1f}, z_{3f}). Interpolate z_1 linearly and z_3 with a cubic polynomial:

$$z_1(s) = z_{1i} + s(z_{1f} - z_{1i}), \tag{5.34}$$
$$z_3(s) = s^3 z_{3f} - (s-1)^3 z_{3i} + \alpha_3 s^2(s-1) + \beta_3 s(s-1)^2, \tag{5.35}$$

which already satisfies the boundary conditions on z_1, z_3. The additional boundary conditions for z_2 are

$$z_{2i} = \frac{z_3'(0)}{z_1'(0)}, \qquad z_{2f} = \frac{z_3'(1)}{z_1'(1)}.$$

Solving for α_3, β_3:

$$\alpha_3 = z_{2f}(z_{1f} - z_{1i}) - 3z_{3f}, \tag{5.36}$$
$$\beta_3 = z_{2i}(z_{1f} - z_{1i}) + 3z_{3i}. \tag{5.37}$$

Once $z_1(s), z_3(s)$ are known, reconstruct $z_2(s), \tilde{w}_1(s), \tilde{w}_2(s)$ through the reconstruction maps. Then $q(s), \tilde{v}(s), \tilde{\omega}(s)$ are obtained by inverting the chained-form coordinate and input transformations.

A singularity arises when $z_{1i} = z_{1f}$, because $z_1'(s) = z_{1f} - z_{1i} = 0$, and z_{2i}, z_{2f} cannot be inferred (division by zero). The fix is to set

$$z_{1f} = z_{1i} + 2\pi,$$

or to add an intermediate configuration $q_m = (x_m, y_m, \theta_m)$ with $\theta_m \neq \theta_i$.

5.6 Path Planning via Parametrized Inputs

A different approach, not based on flatness, writes the geometric inputs in parametrized form and imposes that the robot goes from q_i to q_f. Assume that the robot model is in chained form, eq. (3.22):

$$\begin{cases} z_1' = \tilde{w}_1, \\ z_2' = \tilde{w}_2, \\ z_3' = z_2 \tilde{w}_1, \\ \vdots \\ z_n' = z_{n-1} \tilde{w}_1, \end{cases}$$

which is linear in z when $\tilde{w}_1$ is constant. Let z_i, z_f be the initial and final configurations in chained-form coordinates, and define

$$\Delta = z_{1f} - z_{1i}.$$

Choose the following profiles for the geometric inputs:

$$\begin{cases} \tilde{w}_1 = \text{sign}(\Delta), \\ \tilde{w}_2 = \displaystyle\sum_{j=0}^{n-2} c_j s^j, \end{cases} \qquad s \in [0, |\Delta|].$$

With this choice, $z_2' = \tilde{w}_2$ can be integrated analytically to get the profiles of $z_2, z_3, \ldots, z_n$ as functions of s, and the conditions $z_j(|\Delta|) = z_{jf}$ for each j give a linear system

$$D(\Delta) \begin{pmatrix} c_0 \\ \vdots \\ c_{n-2} \end{pmatrix} = d(z_i, z_f, \Delta).$$

If $D(\Delta)$ is invertible, the coefficients are found. For a 2-input, 3-state system,

$$D = \begin{pmatrix} |\Delta| & \frac{\Delta^2}{2} \\ \operatorname{sign}(\Delta)\,\frac{\Delta^2}{2} & \frac{\Delta^3}{6} \end{pmatrix}, \qquad d = \begin{pmatrix} z_{2f} - z_{2i} \\ z_{3f} - z_{3i} - z_{2i}\Delta \end{pmatrix}.$$

5.7 Choice of the Timing Law

After the geometric path $q(s)$ has been computed, the timing law $s(t)$ must be chosen so that the trajectory is executable by the robot. Since the path is geometric, $s(t)$ is required to be monotonic increasing,

$$\dot{s} > 0,$$

because traversing the path backwards has no meaning (a different direction has to be chosen at the geometric level).

5.7.1 Linear Timing Law

The simplest choice is linear interpolation from s_i to s_f:

$$s(t) = s_i + (s_f - s_i)\frac{t}{T}, \qquad t \in [0, T].$$

The velocity is constant, $\dot{s} = (s_f - s_i)/T$, and the trajectory is the path traversed at constant speed.

5.7.2 Trapezoidal Timing Law

A more realistic choice starts and ends with zero velocity, using a *bang-coast-bang* acceleration profile:

$$\ddot{s}(t) = \begin{cases} a_{\max} & t \in [t_i, t_i + t_m], \\ 0 & t \in (t_i + t_m, t_f - t_m], \\ -a_{\max} & t \in (t_f - t_m, t_f], \end{cases}$$

with $a_{\max} > 0$ and t_m chosen so that the total displacement is $s_f - s_i$. The velocity profile is trapezoidal; if $t_m = 0$ (short displacement) it degenerates to a triangle and the peak velocity $a_{\max} t_m$ is not reached.

5.7.3 Velocity and Acceleration Bounds

The pseudo-velocities are related to $\dot{s}$ by

$$u(t) = \tilde{u}(s(t))\,\dot{s}(t).$$

For the unicycle, with bounds

$$|v| \le v_{\max}, \qquad |\omega| \le \omega_{\max},$$

and a path with geometric inputs $\tilde{v}(s), \tilde{\omega}(s)$, a linear timing law with slope $1/k$,

$$s(t) = s_i + \frac{t}{k}, \qquad t \in [0, k(s_f - s_i)],$$

gives

$$v(t) = \frac{\tilde{v}(s(t))}{k}, \qquad \omega(t) = \frac{\tilde{\omega}(s(t))}{k}.$$

The bounds are satisfied if

$$k = \max \left\{ \frac{\tilde{v}_{\text{peak}}}{v_{\max}}, \frac{\tilde{\omega}_{\text{peak}}}{\omega_{\max}} \right\},$$

where

$$\tilde{v}_{\text{peak}} = \max_{s \in [s_i, s_f]} |\tilde{v}(s)|, \qquad \tilde{\omega}_{\text{peak}} = \max_{s \in [s_i, s_f]} |\tilde{\omega}(s)|.$$

5.8 Takeaways

1. **Path planning** produces a geometric path $q(s)$; **trajectory planning** adds a timing law $s(t)$.
2. The **geometric model** $q' = G(q)\tilde{u}$ describes the admissible tangent vectors to the path.
3. A driftless system is **differentially flat** if there exists a set of flat outputs from which state and inputs can be reconstructed algebraically. For the unicycle and the bicycle, the flat outputs are the Cartesian coordinates (x, y).
4. Path planning via flatness reduces to polynomial interpolation of the flat outputs, with boundary conditions given by the initial and final configurations and the kinematic constraints.
5. For chained-form systems, the flat outputs are the first and the last state variables; the reconstruction requires a careful handling of the singularity when the initial and final first coordinates coincide.
6. **Trajectory planning** adds the timing law $s(t)$ to the geometric path. Linear and trapezoidal (bang-coast-bang) profiles are the standard choices.
7. For the unicycle, the velocity bounds $|v| \leq v_{\max}$ and $|\omega| \leq \omega_{\max}$ translate into a scaling factor k on the timing law, computed from the peak of the geometric inputs.
8. An alternative to flatness for path planning is to parametrize the geometric inputs on the chained-form system and solve the resulting linear system for the coefficients.

Chapter 6

Motion Control

Following a desired trajectory, or reaching a desired configuration, cannot be done by simply applying the inputs computed from the flat outputs: disturbances and modeling errors would accumulate without bound. A feedback action is needed. In previous robotics courses, the dynamic model of manipulators was controlled through joint torques. For wheeled mobile robots we can work at the kinematic level, because the dynamic model is simpler, can be canceled via feedback, and the wheels are equipped with low-level velocity loops that are reliable.

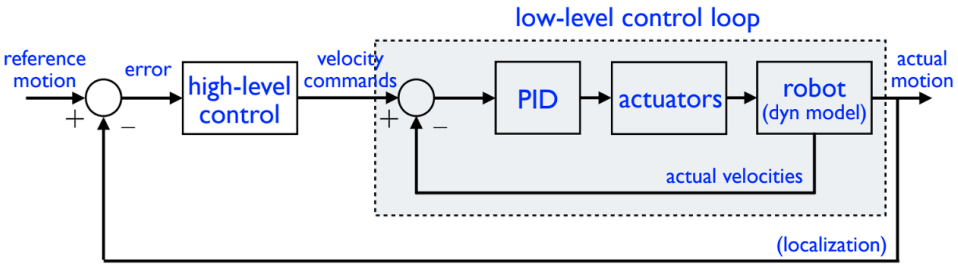


Figure 6.1: Actual control scheme.

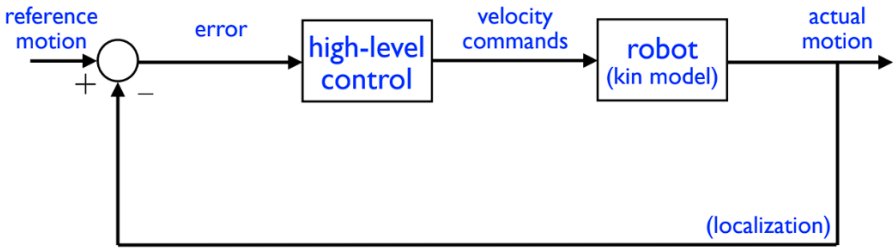


Figure 6.2: Simplified control scheme used for design.

Two control tasks are considered in this course:

- **Trajectory tracking**: the robot must converge to a desired Cartesian trajectory $(x_d(t), y_d(t))$. Treated in this chapter.
- **Posture regulation**: the robot must converge to a desired fixed configuration q_d. Treated in **??**.

Trajectory tracking is more relevant in practice, because moving along planned trajectories reduces the risk of collisions. Posture regulation, on the other hand, does not require any planning.

6.1 Trajectory Tracking for the Unicycle

Let $(x_d(t), y_d(t))$ be an admissible desired trajectory for the unicycle. Admissibility means that there exist inputs v_d, ω_d such that

$$\begin{cases} \dot{x}_d = v_d \cos\theta_d, \\ \dot{y}_d = v_d \sin\theta_d, \\ \dot{\theta}_d = \omega_d. \end{cases} \tag{6.1}$$

Thanks to the flatness property (section 5.3), from the desired trajectory $(x_d(t), y_d(t))$ we can reconstruct the full desired state and the required inputs:

$$\theta_d(t) = \operatorname{atan2}(\dot{y}_d(t), \dot{x}_d(t)) + k\pi, \quad k \in \{0, 1\}, \tag{6.2}$$

$$v_d(t) = \pm\sqrt{\dot{x}_d^2(t) + \dot{y}_d^2(t)}, \tag{6.3}$$

$$\omega_d(t) = \frac{\ddot{y}_d(t)\dot{x}_d(t) - \ddot{x}_d(t)\dot{y}_d(t)}{\dot{x}_d^2(t) + \dot{y}_d^2(t)}. \tag{6.4}$$

An admissible trajectory requires $(\dot{x}_d(t), \dot{y}_d(t))$ to be continuous over time. The desired state trajectory $q_d(t)$ computed from the flat outputs is used to define the state error, and the resulting block diagram is in fig. 6.3.

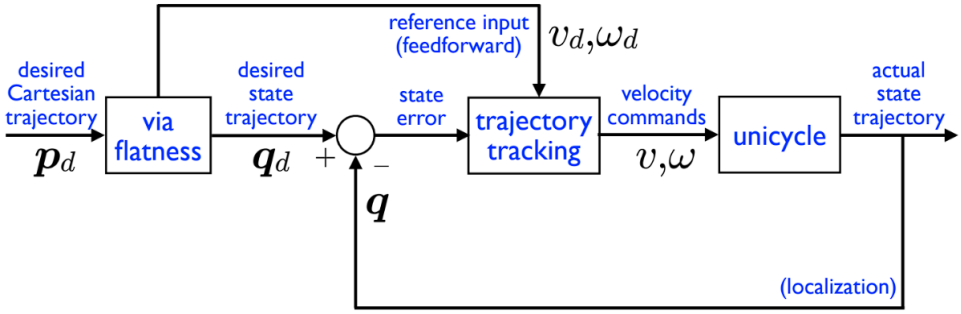


Figure 6.3: Feedforward from flatness, feedback from state error.

6.1.1 Error Dynamics

The objective is to drive the state error to zero. The error is expressed in a frame rotated by θ, with the x-axis parallel to the sagittal axis of the robot:

$$e = \begin{pmatrix} e_1 \\ e_2 \\ e_3 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_d - x \\ y_d - y \\ \theta_d - \theta \end{pmatrix}. \tag{6.5}$$

The pair (e_1, e_2) is the Cartesian error $e_p = (x_d - x,\ y_d - y)^\top$ expressed in the rotated frame, as in fig. 6.4. Differentiating e and using the unicycle kinematic model together with the desired one, we obtain the **error dynamics**:

$$\dot{e}_1 = v_d \cos e_3 - v + e_2\omega, \tag{6.6}$$

$$\dot{e}_2 = v_d \sin e_3 - e_1\omega, \tag{6.7}$$

$$\dot{e}_3 = \omega_d - \omega. \tag{6.8}$$

The error dynamics is **nonlinear** and **time-varying**, since it depends on the reference inputs $v_d(t), \omega_d(t)$.

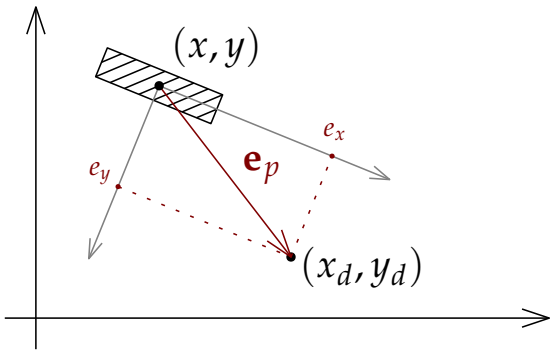


Figure 6.4: Cartesian error expressed in the frame rotated by θ.

6.1.2 Approximate Linearization

Before applying it to the unicycle, recall the idea of stabilization via approximate linearization: to stabilize a nonlinear system at an equilibrium, we stabilize its approximate linearization around it. Consider a generic nonlinear system

$$\dot{x} = \eta(x, u) = f(x) + G(x)u, \qquad x \in \mathbb{R}^n,\ u \in \mathbb{R}^m,$$

and assume, without loss of generality, that the origin is the desired **unforced equilibrium**, i.e. $\eta(0, 0) = 0$, or equivalently $f(0) = 0$. The first-order Taylor expansion of η around $x = 0, u = 0$ is

$$\eta(x, u) \approx \underbrace{\eta(0, 0)}_{=0} + \left.\frac{\partial \eta}{\partial x}\right|_0 x + \left.\frac{\partial \eta}{\partial u}\right|_0 u,$$

so the **approximate linearization** of the system at the origin is

$$\dot{x} = Ax + Bu,$$

where A and B are the two Jacobians evaluated at the origin. With the linear feedback $u = Kx$,

$$\dot{x} = (A + BK)x.$$

If K is chosen so that $A + BK$ is **Hurwitz** (certainly possible if (A, B) is controllable), the approximate linearization is asymptotically stable.

Theorem 6.1 (Lyapunov Indirect Method)**.** *If the approximate linearization of a nonlinear system at an equilibrium is asymptotically stable, then the equilibrium is* ***locally*** *asymptotically stable for the nonlinear system.*

Application to the unicycle. In the form eq. (6.6), the origin is **not** an unforced equilibrium: for $e = 0$, the terms $v_d - v$ and $\omega_d - \omega$ must vanish to have $\dot{e} = 0$. This is fixed by the (invertible) input transformation

$$w_1 = v_d \cos e_3 - v, \tag{6.9}$$
$$w_2 = \omega_d - \omega. \tag{6.10}$$

Substituting $\omega = \omega_d - w_2$, the error dynamics $\dot{e} = \eta(e, w)$ takes the form

$$\dot{e} = \underbrace{\begin{pmatrix} \omega_d e_2 \\ -\omega_d e_1 + v_d \sin e_3 \\ 0 \end{pmatrix}}_{f(e)} + \underbrace{\begin{pmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{pmatrix}}_{G(e)} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}, \tag{6.11}$$

which is at rest for $e = 0, w = 0$, i.e. $\eta(0,0) = 0$. Both the drift and the input term are nonlinear, and the drift is in general time-varying. The Jacobians are

$$\frac{\partial \eta}{\partial e} = \begin{pmatrix} 0 & \omega_d - w_2 & 0 \\ -\omega_d + w_2 & 0 & v_d \cos e_3 \\ 0 & 0 & 0 \end{pmatrix}, \qquad \frac{\partial \eta}{\partial w} = \begin{pmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{pmatrix},$$

and evaluating at $e = 0, w = 0$:

$$A(t) = \begin{pmatrix} 0 & \omega_d & 0 \\ -\omega_d & 0 & v_d \\ 0 & 0 & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}. \tag{6.12}$$

The pair $(A(t), B)$ is controllable iff the matrix

$$\begin{pmatrix} B & AB \end{pmatrix}$$

has full rank. In this case,

$$\operatorname{rank} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -\omega_d & v_d \\ 0 & 1 & 0 & 0 \end{pmatrix} = 3,$$

which holds if $\omega_d \neq 0$ or $v_d \neq 0$. The system loses controllability when the unicycle stops.

Note that A is actually $A(t)$, because the reference inputs v_d, ω_d depend on time: the linear approximation is still a **time-varying** system. Define the linear feedback

$$w = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \underbrace{\begin{pmatrix} -k_1 & 0 & 0 \\ 0 & -k_2 & -k_3 \end{pmatrix}}_{K} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \end{pmatrix} = Ke.$$

The closed-loop error dynamics is

$$\dot{e} = (A(t) + BK)e = A_{cl}(t)e = \begin{pmatrix} -k_1 & \omega_d & 0 \\ -\omega_d & 0 & v_d \\ 0 & -k_2 & -k_3 \end{pmatrix} e,$$

still time-varying. Setting

$$k_1 = k_3 = 2\zeta a, \qquad k_2 = \frac{a^2 - \omega_d^2}{v_d},$$

with $a > 0$ and $\zeta \in (0,1)$, the characteristic polynomial of $A_{cl}(t)$ becomes time-invariant and Hurwitz:

$$p(\lambda) = (\lambda + 2\zeta a)(\lambda^2 + 2\zeta a\lambda + a^2).$$

It has one real negative eigenvalue and two complex eigenvalues with damping ζ and natural frequency a.

Caveats of Approximate Linearization

Having the eigenvalues of $A_{cl}(t)$ fixed at every instant does *not* guarantee asymptotic stability of a time-varying system, unless v_d and ω_d are constant (rectilinear and circular trajectories). Even in that case, asymptotic stability is only *local*, because it comes from theorem 6.1. Moreover, $k_2 \to \infty$ as $v_d \to 0$: the controller can only be used with **persistent** Cartesian trajectories (stops are not allowed).

The **actual** velocity inputs are recovered from the input transformation:

$$v = v_d \cos e_3 - w_1 = v_d \cos e_3 + k_1 e_1, \tag{6.13}$$

$$\omega = \omega_d - w_2 = \omega_d + k_2 e_2 + k_3 e_3. \tag{6.14}$$

As $e \to 0$, $(v, \omega) \to (v_d, \omega_d)$: the control reduces to the pure feedforward term. The controller is **static** and based on the state error. It needs v_d, ω_d (computed via flatness) and θ, both for the error rotation and for the input transformation.

Global stability can be guaranteed by a nonlinear version of the feedback:

$$w_1 = -k_1(v_d, \omega_d) e_1, \tag{6.15}$$

$$w_2 = -k_2 v_d \frac{\sin e_3}{e_3} e_2 - k_3(v_d, \omega_d) e_3, \tag{6.16}$$

with $k_2 > 0$ constant and k_1, k_3 bounded, positive, with bounded derivatives. The actual inputs are recovered from the same input transformation.

6.1.3 Output Error Feedback: Static

A different approach builds the feedback from the **output (Cartesian) error** only, without computing a desired state trajectory $q_d(t)$, using the velocity $\dot{y}_d$ along the reference trajectory as the **feedforward term**. Here $y = (x, y)^\top$ is the Cartesian position of the unicycle and y_d the desired one. The controller is designed by **exact input-output linearization**.

Exact input-output linearization. Consider a driftless nonlinear system with the same number of inputs and outputs:

$$\dot{x} = G(x)u, \qquad x \in \mathbb{R}^n,\ u \in \mathbb{R}^m, \tag{6.17}$$

$$y = h(x), \qquad y \in \mathbb{R}^m. \tag{6.18}$$

Differentiating the output,

$$\dot{y} = \frac{\partial h}{\partial x}\dot{x} = \underbrace{\frac{\partial h}{\partial x} G(x)}_{T(x)} u = T(x)u,$$

where T is an $m \times m$ matrix called the **decoupling matrix**. If T is invertible, the **linearizing feedback**

$$u = T^{-1}(x) w$$

yields $\dot{y} = w$: an **exactly linear** map from the new inputs w to the derivative of the outputs. Given a reference output $y_d(t)$, the output error $e = y_d - y$ evolves as

$$\dot{e} = \dot{y}_d - \dot{y} = \dot{y}_d - w.$$

With the choice $w = \dot{y}_d + Ke$ (feedforward plus proportional feedback),

$$\dot{e} = -Ke,$$

which is **globally exponentially stable** if the eigenvalues of K lie in the right half-plane. The final control law is

$$u = T^{-1}(x)(\dot{y}_d + Ke).$$

Singular decoupling matrix. For the unicycle with output $y = (x, y)^\top$, the decoupling matrix is singular:

$$\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \begin{pmatrix} \cos\theta & 0 \\ \sin\theta & 0 \end{pmatrix} \begin{pmatrix} v \\ \omega \end{pmatrix}.$$

Hence exact input-output linearization is **not possible** for the output (x, y).

Displaced output. The solution is to move the controlled output from the wheel contact point to a point B along the sagittal axis, at distance b:

$$\begin{pmatrix} x_B \\ y_B \end{pmatrix} = \begin{pmatrix} x + b\cos\theta \\ y + b\sin\theta \end{pmatrix}.$$

Differentiating,

$$\begin{pmatrix} \dot{x}_B \\ \dot{y}_B \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -b\sin\theta \\ \sin\theta & b\cos\theta \end{pmatrix}}_{T(\theta),\ \det T = b} \begin{pmatrix} v \\ \omega \end{pmatrix}.$$

If $b \neq 0$, set

$$\begin{pmatrix} v \\ \omega \end{pmatrix} = T^{-1}(\theta) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\dfrac{\sin\theta}{b} & \dfrac{\cos\theta}{b} \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix},$$

obtaining the input-output linearized system

$$\dot{x}_B = w_1, \tag{6.19}$$

$$\dot{y}_B = w_2, \tag{6.20}$$

$$\dot{\theta} = \frac{w_2\cos\theta - w_1\sin\theta}{b}. \tag{6.21}$$

Global exponential convergence of x_B, y_B to the desired trajectory is achieved with

$$w_1 = \dot{x}_d + k_x(x_d - x_B), \tag{6.22}$$

$$w_2 = \dot{y}_d + k_y(y_d - y_B), \tag{6.23}$$

with $k_x, k_y > 0$. The orientation θ is **not** controlled by this scheme: there is a **zero dynamics**, the last equation above. Still, θ must evolve as dictated by flatness.

An advantage of this approach is that the desired trajectory for B can be **arbitrary**: square corners may be included. The controller is **static**, based on the output error, it needs $\dot{y}_d$, and it needs x, y, θ for the output reconstruction and θ for the input transformation.

6.1.4 Output Error Feedback: Dynamic

Rather than displacing the output to B, one can keep the (flat) output $y = (x, y)^\top$ and achieve exact linearization with a **dynamic compensator**. Perform a **dynamic extension** on the driving velocity channel: treat v as a state and its acceleration a as a new input,

$$\dot{v} = a.$$

Now differentiate the output further:

$$\ddot{x} = a\cos\theta - v\sin\theta\,\omega, \tag{6.24}$$

$$\ddot{y} = a\sin\theta + v\cos\theta\,\omega. \tag{6.25}$$

The new decoupling matrix is

$$\begin{pmatrix} \ddot{x} \\ \ddot{y} \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -v\sin\theta \\ \sin\theta & v\cos\theta \end{pmatrix}}_{T(v,\theta),\ \det T = v} \begin{pmatrix} a \\ \omega \end{pmatrix},$$

which is nonsingular if $v \neq 0$. Under this assumption,

$$\begin{pmatrix} a \\ \omega \end{pmatrix} = T^{-1}(v,\theta) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\dfrac{\sin\theta}{v} & \dfrac{\cos\theta}{v} \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix},$$

and the system becomes $\ddot{x} = w_1$, $\ddot{y} = w_2$. **Global exponential convergence** of x, y to the desired trajectory is achieved with

$$w_1 = \ddot{x}_d + k_{p,x}(x_d - x) + k_{d,x}(\dot{x}_d - \dot{x}), \tag{6.26}$$

$$w_2 = \ddot{y}_d + k_{p,y}(y_d - y) + k_{d,y}(\dot{y}_d - \dot{y}), \tag{6.27}$$

with $k_{p,x}, k_{p,y}, k_{d,x}, k_{d,y} > 0$. Since the original flat output is controlled, there is **no zero dynamics** with this approach.

The desired trajectory must be twice differentiable and **persistent**, i.e. $v_d \neq 0$ always (the singularity is structural in nonholonomic systems). The controller is **dynamic** (the velocity v is the state of the compensator, obtained by integrating a), based on the output error. It needs $\dot{y}_d$ and $\ddot{y}_d$, and it needs x, y for the error computation and θ for the input transformation.

6.1.5 Comparison of the Controllers

The three controllers have been compared in simulation while tracking a circle, a square, and a figure eight. The outcomes:

- **Approximate linearization**: only **local** stability, and only for constant v_d, ω_d. It converges to zero error on a circle; on a square it produces a new transient at each corner; on a figure eight local stability is not guaranteed, though the observed performance is good.
- **Static input-output linearization**: the steady-state error norm equals b, because it is the point B that converges to the desired trajectory, while the contact point stays at distance b from it. The displaced output provides a **lookahead** behavior when tracking a square. Reducing b reduces the steady-state error, but the steering velocity increases, since the term $1/b$ appears in T^{-1}.
- **Dynamic input-output linearization**: zero steady-state error, reasonable velocities, and faster transients (for example at the corners of the square).

6.2 Takeaways

1. Motion control for wheeled mobile robots is designed at the **kinematic level**: the dynamic model is canceled by the low-level velocity loops.
2. In **trajectory tracking** the desired Cartesian trajectory is mapped to the desired state and inputs via flatness, and a feedback action is added on the state or output error.
3. **Approximate linearization** of the tracking error gives a linear time-varying error dynamics. Local asymptotic stability follows from the indirect Lyapunov method, but only for constant reference inputs, and the controller requires $v_d \neq 0$.
4. **Exact input-output linearization** is not possible for the Cartesian position of the unicycle: the decoupling matrix is singular. Two fixes: displace the output along the sagittal axis (static feedback, with zero dynamics and steady-state error proportional to the displacement), or extend the dynamics by treating v as a state (dynamic feedback, no zero dynamics, requires $v \neq 0$).
5. All controllers require a **persistent** trajectory: the nonholonomic singularity at $v = 0$ is structural and cannot be removed by feedback.

Chapter 7

Localizzazione

Chapter 8

Mapping

Chapter 9

SLAM

Chapter 10

Decisioni incertezza

Chapter 11

Laboratorio

Appendix A

Formulario

Appendix B

Esercizi

Appendix C

Codice

In questa appendice sono raccolti alcuni frammenti di codice utilizzati durante il corso.

C.1 Esempio: EKF SLAM

Listing C.1: Implementazione EKF SLAM

C.2 Esempio: RRT

Listing C.2: Implementazione RRT

Bibliography

[Cho+05] Howie Choset et al. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, 2005.

[Sic+25] Bruno Siciliano et al. *Foundations of Robotics*. Springer, 2025.